RTL Design Sherpa

DDR2 / LPDDR2 Family Controller Micro-Architecture Specification 0.1

June 13, 2026

Table of Contents

List of Figures

No figures in this document.

List of Tables

No tables in this document.

1 Ddr2 Lpddr2 Mas Index

Generated: 2026-06-16

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

2 Document Information

2.1 DDR2 / LPDDR2 Family Controller Micro-Architecture Specification

Document Number: DDR2-LPDDR2-MAS-001 **Version:** 0.1 **Status:** Draft - Skeleton **Classification:** Open Source - Apache 2.0 License

---

2.2 Document Purpose

This Micro-Architecture Specification (MAS) is the implementation-level companion to the DDR2 / LPDDR2 Hardware Architecture Specification (HAS). Where the HAS answers “what is the architecture and why,” this MAS answers “what is in the RTL and how does it work.”

The MAS is the document RTL designers, verification engineers, and integrators consult when writing or reviewing SystemVerilog code, building testbenches, and stitching the controller into an SoC.

Target Audience:

- RTL designers implementing the FUBs described in this document
- Verification engineers writing per-FUB cocotb tests
- Integrators stitching the controller into an SoC
- Software engineers writing low-level memory configuration code (driver authors, bring-up engineers)

Companion Documents:

- DDR2/LPDDR2 Family Controller HAS (`../ddr2_lpddr2_has/`) — high-level architecture and design rationale

- DDR2/LPDDR2 DFI BFM documentation (in `RTLDesignSherpa-DV`) — verification-side reference

2.3 Document Scope

The MAS is the **micro-architecture** view: per-FUB inputs/outputs, internal state, FSM diagrams, datapath timing, register-level pipeline stages, and per-block timing budgets. It assumes the reader has read the HAS for context.

This document covers:

- Top-level integration wiring (`ddr2_lpddr2_ctrl`)
- Each FUB's interface signal list, internal storage, datapath flow, FSM, and timing budget
- AXI4 and DFI v2.1 wire-level protocol details specific to this controller
- APB programming interface and the full CSR register map
- Programming sequences (init, refresh, power-down, multi-rank, error handling)
- Build-time and runtime configuration reference

This document does **not** cover:

- Higher-level architectural rationale (see HAS)
- Verification plans or coverage models (see `dv/README.md`)
- Floorplan or layout guidance (project-specific)
- Bit-level CSR YAML — that lives in `regs/` and is the source of truth for CSR generation

2.4 Revision History

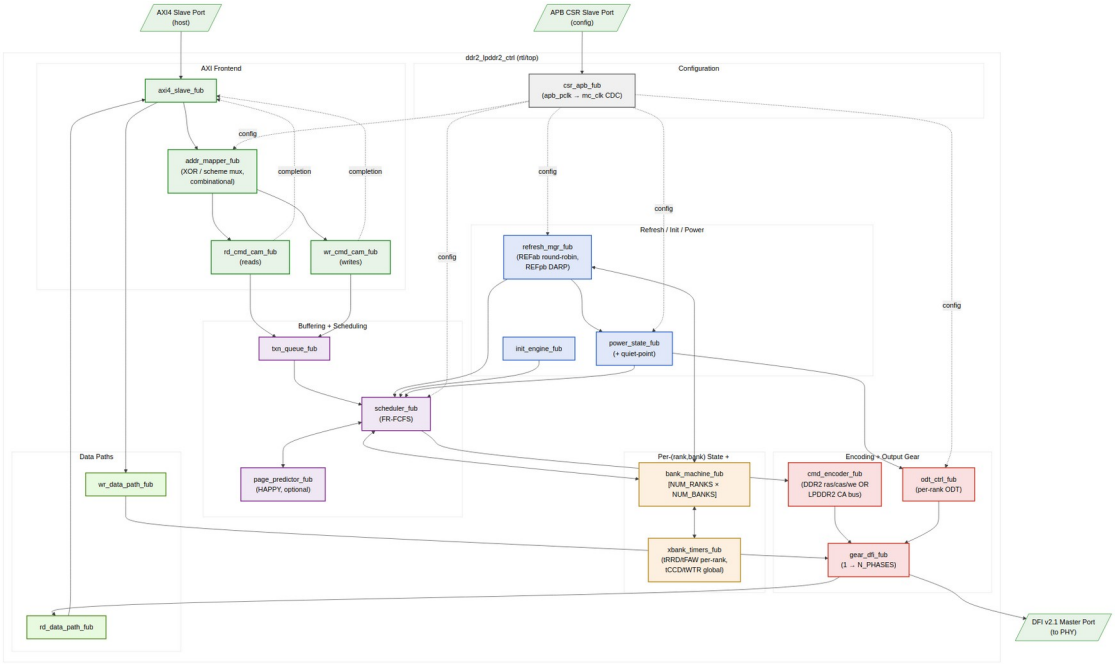
Version	Date	Author	Notes
0.1	2026-06-14	RTL Design Sherpa	Initial skeleton — chapter outline, FUB list, port list scaffold

3 Architecture and Datapath

This chapter is the implementation-level architectural orientation: the top-level FUB stitching, the read/write datapath flow, and the clocking topology. Per-FUB detail is in §2.

3.1 Top-Level FUB Topology

The controller's top is `ddr2_lpddr2_ctrl.sv` — a pure structural integration of leaf FUBs. The 18 leaf FUBs are partitioned into 7 functional groups (see HAS §2.5 for the SWAG-level group description; this section is the wire-level topology view).



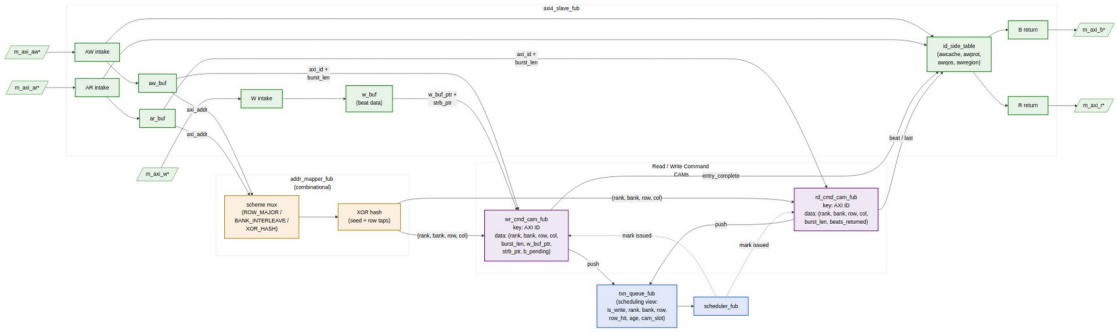
Top-Level FUB Topology

Source: 01_top_fub_topology.mmd

The salient property: there is exactly **one** stage where AXI-layer concepts (burst, ID, write strobe) cross into DRAM-layer concepts (rank, bank, row, column). That stage is `addr_mapper` → `{rd,wr}_cmd_cam`. Upstream of `addr_mapper` everything is AXI; downstream of the CAMs everything is DRAM.

3.2 Read / Write Datapath: AXI → addr-hash → CAM

The user-facing entry point is `axi4_slave_fub`. From the slave, the read and write paths share `addr_mapper` (one cycle) and then split into two separate CAMs:



AXI → addr_mapper → CAMs Datapath

Source: 02_axi_to_cam_datapath.mmd

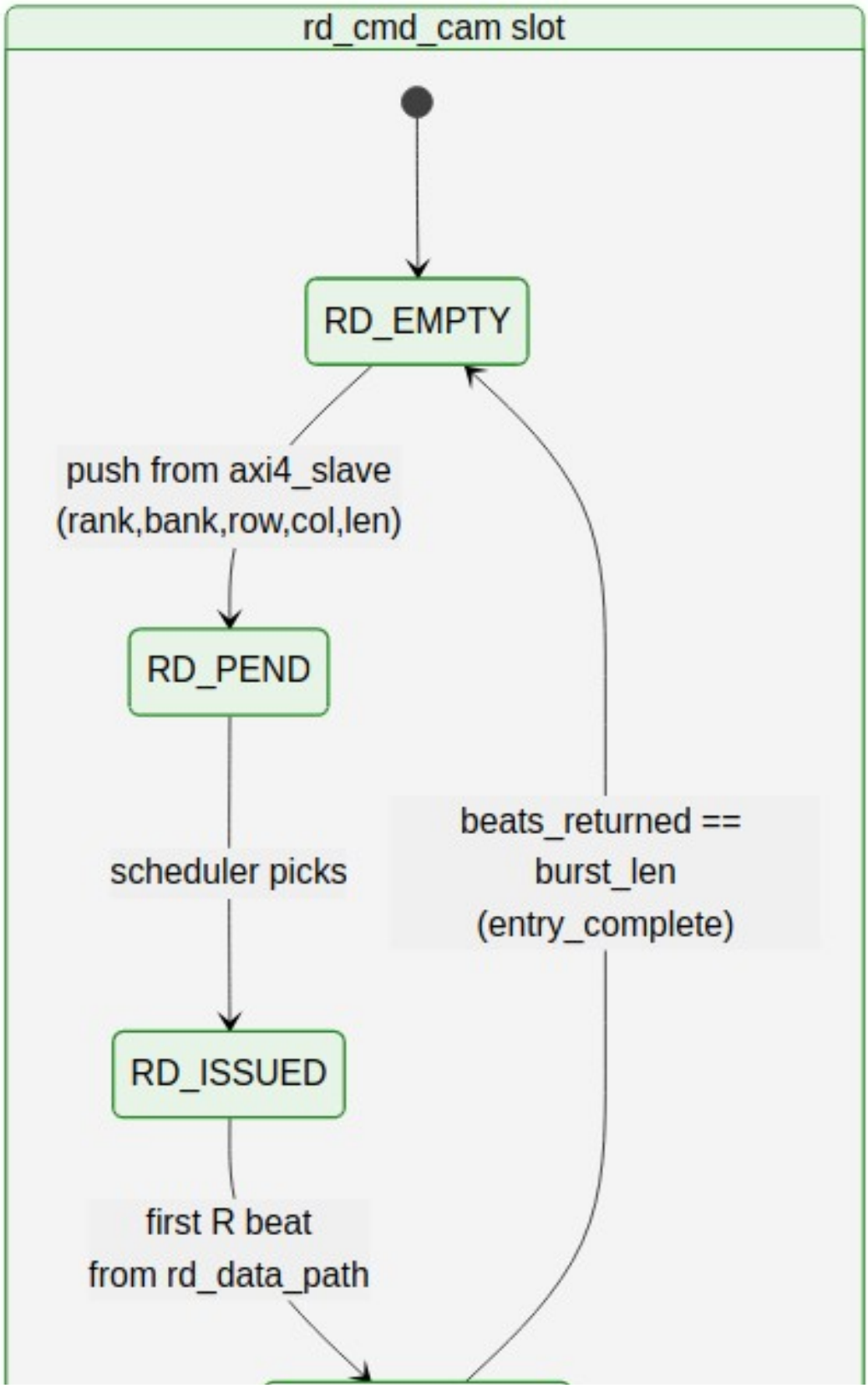
3.2.1 Why Two CAMs, Not One

The CAM split happens here, not at the scheduler, because read and write CAMs hold **different metadata** and have **different lifetimes**:

CAM	Key	Data	Cleared on
`wr_cmd_cam`	AXI ID	(rank, bank, row, col, burst_len, **w_buf_ptr**, **strb_ptr**)	B-channel response push
`rd_cmd_cam`	AXI ID	(rank, bank, row, col, burst_len, **rid_counter**, **expected_beats**)	last R beat of the burst

A single unified CAM would either carry both metadata sets (wasteful) or force an awkward "is_write" predicate on every lookup. Keeping the CAMs separate also means the read and write paths can be sized independently — the read path typically deeper because read latency is longer; the write path can be shallower because write completion is fire-and-forget.

The two CAMs have visibly different lifecycles — note in particular that the write side stays "alive" through the post-issue write window (waiting on `b_complete` from `xbank_timers`) while the read side just counts beats:


rd_cmd_cam slot
RD_EMPTY
push from axi4_slave
(rank,bank,row,col,len)
RD_PEND
scheduler picks
RD_ISSUED
first R beat
from rd_data_path
beats_returned ==
burst_len
(entry_complete)

CAM Slot Lifecycles (read vs. write)

Source: 03_cam_lifecycles.mmd

3.2.2 Why the XOR Happens Before the CAM

Address XOR / hash (`addr_mapper`) **must** happen before the CAM push because the CAM stores the post-decode (rank, bank, row, col) tuple — not the raw AXI address. Two reasons:

1. **Stable CAM key data.** The CAM is queried by the scheduler in the (rank, bank) dimension — for example, "do any pending writes target rank 0 bank 3, row R?" Storing the post-decode tuple makes this a direct comparison; storing the raw AXI address would force the scheduler to re-XOR on every match.

2. **Address-mapping scheme is per-controller, not per-burst.** The `ADDR_MAP_TUNING.scheme_or` field is global to the controller. If we stored raw addresses, a runtime scheme change (which can happen at any quiet point) would force a CAM walk to re-decode every entry. Storing the decoded tuple means a scheme change only affects **new** AXI bursts — old in-flight bursts continue with their decoded address. This matches the quiet-point semantics in HAS §6.3.

The `addr_mapper` is **combinational** — no pipeline stage between AXI intake and CAM push — so the timing budget for the XOR network is one cycle minus the AXI register-slice slack. See `ch02_blocks/03_addr_mapper.md` for the timing breakdown.

3.2.3 Why the AXI-side metadata stays in `axi4_slave_fub`

Some AXI fields (`awcache`, `awprot`, `awqos`, `awregion`, `bid`/`rid`) never need to cross into DRAM-layer state. These are held in small **side tables** inside `axi4_slave_fub`, indexed by the same AXI ID that keys the CAMs. The CAM holds the *DRAM* state; the side table holds the *AXI* state. On B/R completion, the CAM produces the completion signal and the side table produces the response metadata; the two are joined at the AXI return port.

This split keeps the CAM narrow (16 ID slots × ~36 bits in the default config) and pushes the AXI-specific clutter into the FUB that already owns the protocol.

3.3 Clocking Topology

Two external clocks; one CDC.

Clock	Frequency Range	Drives
`mc_clk`	100–500 MHz	All DRAM-layer FUBs: scheduler, bank machines, refresh, init, power, cmd_encoder, gear_dfi, odt_ctrl, data paths, and `axi4_slave`
`apb_pclk`	25–100 MHz	CSR slave only

Clock	Frequency Range	Drives

`csr_apb_fub` owns the single CDC between `apb_pclk` and `mc_clk`. CSR overrides are handed off across this CDC into a quiet-point staging register; the actual override-application happens at the next quiet point on `mc_clk`. Quiet-point detection is in `power_state_fub` (no commands in flight, no refresh in progress).

There is **no CDC** in the AXI4 datapath — `axi4_slave_fub` runs in the `mc_clk` domain. If the SoC's AXI master is on a different clock, an external clock converter is required upstream of the controller.

3.4 DFI Phase / Gear Topology

The DFI v2.1 master interface is `N_PHASES`-deep. The MC clock drives one frame of `N_PHASES` DFI cycles per MC clock cycle. The `gear_dfi_fub` rewinds scheduler-issued commands and AXI data beats into the right per-phase slot per `WRPHASE` / `RDPHASE`. The scheduler does not see the phase dimension — its issue rate is one command per MC clock; the gear absorbs all of the phase-level scheduling.

The block diagrams and FSM diagrams referenced inline above are in `assets/mermaid/` (TBD; placeholders until the per-block detail in §2 is complete).

---

4 Top-Level Port List

This chapter is the **wire-level** port list of the `ddr2_lpddr2_ctrl` top module — the exact signal names, directions, widths, and parameter dependencies that the integration script and SoC top-level use.

The implementation-level signal list is the same as the HAS overview view (HAS §2.4); this chapter restates it in the SystemVerilog-derivable form, with notes on bit-ordering, default tie-offs, and synthesis attributes where they matter.

The canonical machine-generated source is `regs/ddr2_lpddr2_ctrl_ports.svh` — what follows is the human-readable mirror.

4.1 Parameter-Dependent Widths (re-stated)

- `AW` = `AXI_ADDR_WIDTH`
- `DW` = `AXI_DATA_WIDTH`
- `SW` = `DW/8`
- `IDW` = `AXI_ID_WIDTH`
- `DFI_DW` = `DFI_DATA_WIDTH`
- `DFI_AW` = `DFI_ADDR_WIDTH`
- `NP` = `N_PHASES`
- `NR` = `NUM_RANKS`
- `NB` = `NUM_BANKS`
- `BA` = `$clog2(NUM_BANKS)`

4.2 AXI4 Slave Port List

See HAS §2.4 §1 for the table. The MAS adds nothing beyond what the HAS already published; the SystemVerilog declaration uses the same field names directly. AXI side-band signals (`awqos`, `awregion`, `arqos`, `arregion`) are observed but do not currently drive scheduler priority in v1; the wiring exists for the v2 QoS feature.

4.3 DFI v2.1 Master Port List

See HAS §2.4 §2. Additional MAS-level notes:

- The `dfi_cs_n[NR]` packed dimension is **outer-most** in SV declaration: `output [NP-1:0][NR-1:0] dfi_cs_n;`
- Per-rank `dfi_cke[NR]` is **outer-most**: `output [NR-1:0] dfi_cke;`
- The DDR2 ras/cas/we strobes are present even in LPDDR2 builds (driven all-1s, tied off by synthesis); the elaboration-time `MEMTYPE` parameter does *not* remove them from the port list, so the same SV header is usable for both flavors.

4.4 APB CSR Slave Port List

See HAS §2.4 §3. The APB clock and reset (`apb_pclk`, `apb_presetn`) are in the misc port group below; only the protocol signals are in this APB block.

4.5 Clocks, Resets, IRQs, Debug

See HAS §2.4 §4.

Additional MAS notes:

- `mc_rst_n` is asynchronous-assert, synchronous-deassert inside the controller (the deassert synchronizer lives in `power_state_fub`'s reset domain).
- `apb_presetn` is independently synchronized in `csr_apb_fub`.
- `irq_*` outputs are asserted in the `mc_clk` domain; the SoC's interrupt controller is responsible for re-synchronizing if it runs on a different clock.

4.6 SystemVerilog Header Snippet

The first 30 lines of the generated port header — the rest is per the tables above:

```
module ddr2_lpddr2_ctrl #(
    parameter string MEMTYPE          = "DDR2",
    parameter int    N_PHASES         = 4,
    parameter int    NUM_RANKS        = 1,
    parameter int    NUM_BANKS        = 8,
    parameter int    AXI_DATA_WIDTH   = 64,
    parameter int    AXI_ID_WIDTH     = 4,
    parameter int    AXI_ADDR_WIDTH   = 32,
    parameter int    ROW_WIDTH        = 14,
    parameter int    COL_WIDTH        = 10,
    parameter int    DFI_DATA_WIDTH   = 32,
    parameter int    DFI_ADDR_WIDTH   = 14,
    parameter int    WRPHASE          = 0,
    parameter int    RDPHASE          = 0,
    parameter int    TXN_QUEUE_DEPTH  = 16,
    parameter string SCHEDULER_MODE   = "OOO",
    parameter int    LOOKAHEAD_DEPTH_MAX = 4,
    parameter string PAGE_POLICY      = "HAPPY_HYBRID",
    parameter int    PAGE_PREDICTOR_TABLE_BITS = 12,
    parameter int    AGE_MAX          = 256,
    parameter int    REFRESH_DEFER_MAX = 8,
    parameter string REFPB_POLICY     = "DARP",
    parameter string ODT_RULE_MULTIRANK = "JEDEC_DDR2",
    parameter int    SIM_INIT_SCALE   = 1
) (
    // Clocks and resets
    input  logic                        mc_clk,
    input  logic                        mc_rst_n,
    input  logic                        apb_pclk,
    input  logic                        apb_presetn,
```

```
    // AXI4 slave, DFI master, APB slave, IRQs, debug — see chapter
tables.

    // ...
);
```

The full module declaration is in `regs/ddr2_lpddr2_ctrl_ports.svh` (machine-generated).

---

5 Clocks and Reset

5.1 Clock Domains

Clock	Frequency	Polarity	Domain Members
`mc_clk`	100–500 MHz	Posedge	All DRAM-layer FUBs, `axi4_slave`, data paths
`apb_pclk`	25–100 MHz	Posedge	`csr_apb_fub` only

The DFI v2.1 interface is driven on `mc_clk` (the controller side of DFI is always controller-clock; the DFI PHY runs at the DRAM clock and the gear mapping is the gear FUB's responsibility).

5.2 Reset Topology

Reset	Polarity	Type	Domain
`mc_rst_n`	Active-low	Async assert, sync deassert	`mc_clk`
`apb_presetn`	Active-low	Async assert, sync deassert	`apb_pclk`

Each reset has its own 2-flop synchronizer for the deassert edge, instantiated in the FUB that owns the domain:

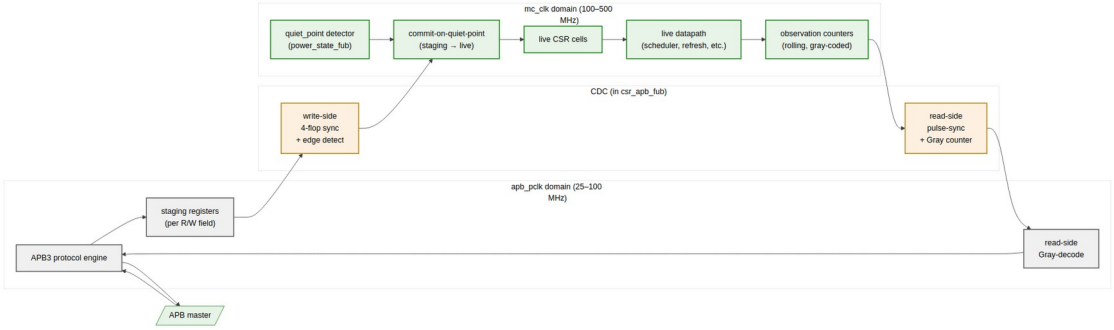
- `mc_rst_n` sync — in `power_state_fub`
- `apb_presetn` sync — in `csr_apb_fub`

Both synchronizers are clear-on-async-assert, hold-during-async-low. No reset-glitch filter is in the controller — the SoC's PMU is expected to drive a clean reset.

5.3 CDC

Exactly one CDC in the controller: APB → MC for CSR overrides and MC → APB for CSR readback. Both are handled in `csr_apb_fub`. The two directions use different mechanisms:

- **APB → MC** (write side): 4-flop synchronizer per register field, with a per-field `apb_write_strobe` edge-detect on the MC side to latch the new value into the staging register. Staging held until next quiet point.
- **MC → APB** (read side): per-counter pulse synchronizers feed Gray-coded saturating counters; APB reads sample the Gray code and decode locally. The MC-side counters never wrap synchronously between reads.



Clock Domains and CDC Topology

Source: 04_clocks_cdc.mmd

5.4 Quiet Points

A **configuration quiet point** is the only safe moment to propagate CSR overrides from staging registers into the live datapath. Quiet points are defined by:

- `txn_queue_fub` is empty *or* all pending entries are in `PENDING` state (no `ISSUED`/`COMPLETING`)
- `refresh_mgr_fub` is in IDLE (not WAIT_BANK_GNTS, DO_REFRESH, DO_ZQCS)
- `power_state_fub` is in ACTIVE (not transitioning)
- No DFI command is in flight (the encoder's per-phase pipeline has drained)

When all four hold, `power_state_fub` asserts `quiet_point`. Override-staging in `csr_apb_fub` watches this signal and commits all queued overrides in one MC-clock edge.

Software triggers quiet-point drain via `CTRL.config_apply`. The CSR slave returns `STATUS.config_settled` when commit is complete.

5.5 Reset Sequence

On power-on:

1. `mc_rst_n` and `apb_presetn` are asserted (both low). PHY drives `dfi_init_complete = 0`.
2. SoC deasserts `apb_presetn`. `csr_apb_fub` comes out of reset; CSRs are R/W-able. Software loads timing CSRs (MR0..MR3, timings, address-map scheme, refresh tuning).
3. SoC deasserts `mc_rst_n`. All DRAM-layer FUBs come out of reset. Controller idles in **POST_RESET** state.
4. Software writes `CTRL.init_start = 1`. `init_engine_fub` walks the per-memtype step table, driving `dfi_reset_n`, MR loads, ZQCAL, etc.
5. On completion, `init_engine_fub` asserts `irq_init_done` (one cycle) and `STATUS.init_done = 1`. Controller transitions to ACTIVE.
6. AXI traffic from the host is honored.

The minimum gap between `apb_presetn` and `mc_rst_n` deassertion is 16 `mc_clk` cycles (the synchronizer chain depth). Software does not need to wait — the controller stalls at POST_RESET until both resets are released.

---

6 Top-Level Integration (`ddr2_lpddr2_ctrl`)

Module: `ddr2_lpddr2_ctrl.sv` **Location:** `rtl/top/` **Category:** TOP (Structural only — no behavioral logic) **Status:** Skeleton

---

6.1 Purpose

`ddr2_lpddr2_ctrl` is the controller's top-level integration. It is **pure structural wiring** — every behavioral statement belongs in a FUB. The role of the top is:

1. Declare the external port list (see §1.2)
2. Instantiate every FUB
3. Wire FUB↔FUB interfaces
4. Fan-out per-rank / per-bank / per-phase signals where the FUB count is parametric

6.2 Instantiation Inventory

Instance Name	FUB	Count	Notes
`u_axi4_slave`	`axi4_slave_fub`	1	Single AXI4 slave port
`u_addr_mapper`	`addr_mapper_fub`	1	Combinational; no clock
`u_rd_cmd_cam`	`rd_cmd_cam_fub`	1	Read-side CAM
`u_wr_cmd_cam`	`wr_cmd_cam_fub`	1	Write-side CAM
`u_txn_queue`	`txn_queue_fub`	1	Unified pending queue
`u_scheduler`	`scheduler_fub`	1	
`u_page_predictor`	`page_predictor_fub`	0 or 1	Conditional on `PAGE_POLICY == HAPPY_HYBRID`
`u_bank_machine[r][b]`	`bank_machine_fub`	`NUM_RANKS × NUM_BANKS`	`generate` block, two-dimensional
`u_xbank_timers`	`xbank_timers_fub`	1	Internally per-rank tRRD/tFAW + global tCCD
`u_refresh_mgr`	`refresh_mgr_fub`	1	
`u_init_engine`	`init_engine_fub`	1	
`u_power_state`	`power_state_fub`	1	
`u_cmd_encoder`	`cmd_encoder_fub`	1	Parameterized by `MEMTYPE`
`u_gear_dfi`	`gear_dfi_fub`	1	
`u_odt_ctrl`	`odt_ctrl_fub`	1	
`u_wr_data_path`	`wr_data_path_fub`	1	
`u_rd_data_path`	`rd_data_path_fub`	1	

Instance Name	FUB	Count	Notes
u_csr_apb	csr_apb_fub	1	Only FUB in apb_pclk domain

6.3 Generate Blocks

The only parametric-fanout instances are the bank machines, fan-out arrays of per-rank CSRs, and per-rank ODT/CKE outputs:

```
generate
    for (genvar r = 0; r < NUM_RANKS; r++) begin : g_rank
        for (genvar b = 0; b < NUM_BANKS; b++) begin : g_bank
            bank_machine_fub #(.ROW_WIDTH(ROW_WIDTH)) u_bank_machine (
                .mc_clk          (mc_clk),
                .mc_rst_n        (mc_rst_n),
                .scheduler_if    (sched_to_bank[r][b]),
                .refresh_if      (refresh_to_bank[r][b]),
                .xbank_if        (xbank_to_bank[r][b]),
                .state_o         (bank_state[r][b]),
                .open_row_o      (bank_open_row[r][b]),
                .last_ref_age_o (bank_last_ref_age[r][b])
            );
        end
    end
endgenerate
```

The `for (genvar r)` outer loop is the per-rank dimension; the inner is the per-bank dimension. The aggregation of `bank_state[r][b]` into the scheduler input is a structural concatenation only — no behavioral aggregation in the top.

6.4 What the Top Does Not Do

- No combinational logic beyond signal renaming or pure concatenation
- No CSR field expansion (the CSR slave handles its own field encoding)
- No clock gating (clock-gate insertion is a synthesis-script concern, not RTL)
- No reset synchronization (handled per-domain in `power_state_fub` and `csr_apb_fub`)
- No `assign` statements except for tying-off optional DFI signals (DDR2 vs LPDDR2 strobe ties)

This intentional poverty of the top file makes it the obvious place for the structural-only synthesis sanity check: “every line in this file is either an instantiation or a port-mapping; if a line is anything else, it is a bug.”

7 AXI4 Slave (axi4_slave_fub)

Module: axi4_slave_fub.sv **Location:** rtl/fub/ **Category:** FUB **Parent:** ddr2_lpddr2_ctrl
Status: Skeleton

7.1 Purpose

axi4_slave_fub is the AXI4 host-facing protocol engine. It accepts AW/W/AR transactions from the SoC, hands off the address to addr_mapper_fub for decode, and pushes the resulting transaction into the appropriate CAM (wr_cmd_cam_fub or rd_cmd_cam_fub). On B and R returns, it pulls completion signals from the CAMs and merges them with AXI-side metadata to produce protocol-compliant responses.

The FUB owns the AXI ID side table — the per-ID set of fields (awcache, awprot, awqos, awregion, bid echo, rid echo) that never need to cross into the DRAM-layer state.

7.2 External Interface (AXI side)

Per HAS §2.4 §1 — full AW/W/B/AR/R channel signal list. No additions in MAS.

7.3 Internal Buffers

Buffer	Depth	Width	Purpose
aw_buf	TXN_QUEUE_DEPTH/2 (default 8)	AW + IDW + 8(len) + 3(size) + 2(burst) + 4(cache) + 3(prot) + 4(qos) + 4(region)	Pending AW until CAM push
w_buf	TXN_QUEUE_DEPTH × max(burst_len)	DW + SW + 1(last) per beat	Write data staging
ar_buf	TXN_QUEUE_DEPTH/2 (default 8)	same AW fields	Pending AR until CAM push
b_response_fifo	small (4)	IDW + 2(resp)	Pending B responses

Buffer	Depth	Width	Purpose
r_response_fifo	small (4)	IDW + DW + 2(resp) + 1(last) per beat	Pending R responses
id_side_table	2^IDW	4(cache) + 3(prot) + 4(qos) + 4(region)	Per-ID AXI metadata, indexed by awid/arid

The buffers above are FUB-internal; they do **not** appear in the architecture-level transaction queue (txn_queue_fub). The transaction queue holds the DRAM-layer view of pending work; these buffers hold the AXI-layer view of in-flight bursts.

7.4 Data Flow

7.4.1 Write Path

1. AW intake: capture aw* fields; push to aw_buf. Compute outstanding-count and assert awready if aw_buf not full.
2. W intake: accept w* beats into w_buf. Each W beat carries wstrb and wlast. The W path does not wait for AW — W can lead AW or trail it within AXI ordering rules.
3. CAM push: when both aw_buf head and the matching w_buf head are present (or when a write-only / address-only burst's prerequisites are satisfied), push to wr_cmd_cam_fub with:
   - key = awid
   - data = (rank, bank, row, col, burst_len, w_buf_ptr, strb_ptr)
   - where (rank, bank, row, col) come from addr_mapper_fub (combinational, same cycle)
4. Scheduler issue: when scheduler picks this entry, the FUB hands w_data_path_fub the w_buf_ptr and strb_ptr for it to walk the data beats.
5. B response: when wr_cmd_cam_fub reports the entry complete (last W beat issued + tDFI write to DFI), the FUB pops the AXI ID, joins with id_side_table[id].axi_metadata, and pushes b_response_fifo.
6. B emit: standard B-channel handshake; pop b_response_fifo.

7.4.2 Read Path

1. AR intake: capture ar* fields; push to ar_buf. Assert arready if not full.
2. CAM push: pop ar_buf head; push to rd_cmd_cam_fub with:
   - key = arid

- `data = (rank, bank, row, col, burst_len, rid_counter, expected_beats)`
- where (rank, bank, row, col) come from `addr_mapper_fub`
- `rid_counter` is the AXI ID echo

3. Scheduler issue: when scheduler picks this entry, it commands the issue of RD / RDA. `rd_data_path_fub` watches for the corresponding DFI rddata beats.
4. R response: as each R beat completes, `rd_data_path_fub` strobes `rd_cmd_cam_fub` to decrement `expected_beats`; the CAM emits `r_beat_to_axi` with `rid`, beat data, `rlast` when the last beat arrives.
5. R emit: standard R-channel handshake; pop `r_response_fifo`.

7.5 Outstanding Burst Tracking

The FUB enforces:

Limit	Constraint
Total in-flight reads	≤ `rd_cmd_cam_fub` depth (default 16)
Total in-flight writes	≤ `wr_cmd_cam_fub` depth (default 16)
Per-ID in-flight reads	≤ `MAX_PER_ID_READS` parameter (default 4)
Per-ID in-flight writes	≤ `MAX_PER_ID_WRITES` parameter (default 4)

If `AXI_OOO_ACROSS_IDS == false`, the per-ID counts reduce to "one in-flight per ID" — the AXI completion is strictly in-order per ID, but the issue order at the scheduler is still OoO.

7.6 Timing Budget

Path	Budget
`awvalid` → `awready` (combinational)	≤ 0.7 of cycle
`aw_buf` push to `addr_mapper` valid	0 cycles (combinational)
`addr_mapper` valid to CAM push	1 cycle
`wr_cmd_cam` push to scheduler-visible	1 cycle
Worst path: `awvalid` → CAM-push register	2 cycles

The two-cycle AXI→CAM latency is the limit on AXI sustained issue rate; with default config it allows 1 burst per 2 cycles, well within the AXI burst rate at 200 MHz.

7.7 CSR Hooks

- `STATUS.axi_inflight_rd` (R only) — current count of in-flight read bursts
- `STATUS.axi_inflight_wr` (R only) — current count of in-flight write bursts
- `STATUS.axi_id_table_occ` (R only) — number of distinct AXI IDs currently in the side table
- `OBS_AXI_R_LATENCY_AVG`, `OBS_AXI_W_LATENCY_AVG` — driven from R/B-channel sampling in this FUB

7.8 TODO (week-of-MAS work)

- FSM diagram for the AW/W join and the AR intake (mermaid)
- B/R response-ordering corner cases (per HAS §3.1 OoO-across-IDs)
- Waveform examples (wavedrom) for back-pressure scenarios
- Detailed table of `id_side_table` field widths and reset values

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

8 Address XOR / Hash (`addr_mapper_fub`)

Module: `addr_mapper_fub.sv` **Location:** `rtl/fub/` **Category:** FUB (combinational; no clock)
Parent: `ddr2_lpddr2_ctrl` **Status:** Skeleton

8.1 Purpose

`addr_mapper_fub` decodes a flat `AXI_ADDR_WIDTH`-bit address into the DRAM-layer (`rank, bank, row, column`) tuple per the currently-active address-mapping scheme. The XOR / hash happens **here** — upstream of the read/write CAMs — so that the CAMs store the decoded tuple and not the raw AXI address.

The FUB is **pure combinational** — there is no clock, no flop, no FSM. The output is valid the same cycle as the input. The path is one of the timing-critical signatures in the design and is treated as a multi-cycle path only if synthesis cannot meet timing.

8.2 Mirror to the Python AddressMapping Class

This RTL is the mirror of the AddressMapping Python class in the DV repository (CocoTBFramework.components.dfi.address_mapping). Both produce **bit-for-bit identical** decoded outputs for the same input. The mirror is enforced by:

- Same scheme list (ROW_MAJOR, BANK_INTERLEAVE, XOR_HASH)
- Same field-bit positions per scheme
- Same XOR_HASH seed and tap matrix

The shared decode function eliminates a common class of bugs where simulation passes (because the BFM decodes one way) but RTL fails (because the controller decodes a different way).

8.3 Interface

Signal	Direction	Width	Description
axi_addr_i	input	AW	Flat AXI byte address
scheme_active_i	input	2	Runtime scheme select from ADDR_MAP_TUNING.scheme_or
xor_seed_i	input	8	Runtime seed for XOR_HASH (from CSR)
bg_field_pos_i	input	3	Bank-group field bit position (unused in DDR2/LPDDR2; reserved)
rank_o	output	$clog2(NR)	Decoded rank
bank_o	output	BA	Decoded bank
row_o	output	ROW_WIDTH	Decoded row
col_o	output	COL_WIDTH	Decoded column (byte-aligned within burst)

8.4 Scheme Decode

8.4.1 Scheme 1: ROW_MAJOR

The simplest mapping; one contiguous row per rank/bank stride. Bit layout:

```
| ... reserved ... | rank | row [ROW_WIDTH-1:0] | bank [BA-1:0] | col [COL_WIDTH-1:0] | byte_in_dq |
```

The byte-in-DQ field is the controller's column-byte offset and is consumed by wr_data_path_fub for wstrb masking; it is **not** part of the decoded col_o (which is column-word aligned).

ROW_MAJOR is best for sequential streaming workloads where a long burst stays in one row.

8.4.2 Scheme 2: BANK_INTERLEAVE

```
| ... reserved ... | rank | row [ROW_WIDTH-1:0] | col_hi [...] | bank
[BA-1:0] | col_lo [...] | byte_in_dq |
```

Bank bits are placed **between** column-low and column-high bits, so consecutive cache lines round-robin across banks. This is the recommended default for general-purpose workloads with mixed access patterns; it produces higher bank-parallelism in the scheduler.

8.4.3 Scheme 3: XOR_HASH

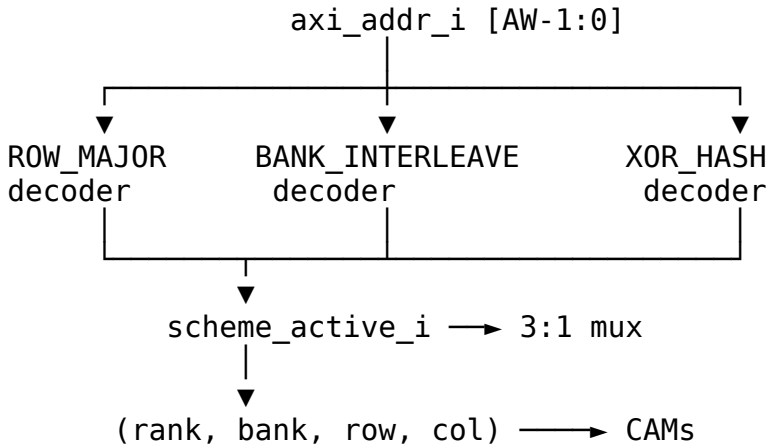
Same layout as BANK_INTERLEAVE, but the bank field and (a subset of) the column-high bits are XOR-hashed with row bits:

```
bank_hashed[i] = bank_raw[i] XOR row_raw[i] XOR row_raw[i + BA] XOR
xor_seed[i]
```

The XOR_HASH scheme defeats pathological row-conflict patterns (e.g., a power-of-two stride that lands on the same bank in BANK_INTERLEAVE). The seed `xor_seed_i` is runtime-modifiable so a bring-up engineer can change the hash without rebuilding.

8.5 Runtime Scheme Mux

The output is multiplexed across the synthesized schemes per `scheme_active_i`. Only schemes in `ADDR_MAP_SCHEMES_SYNTH` are routed to the mux; un-synthesized schemes tie off and writing them via CSR returns `pslverr`.



8.6 Timing Budget

The decoder is purely combinational. The longest path in XOR_HASH is the XOR of (a) several address bits, (b) the seed, and (c) the row bits — about 5–6 levels of XOR gates plus a 3:1 mux. At 500 MHz this is comfortably within 0.5 ns of MC clock cycle time.

Path	Estimate (gate-level)
`axi_addr_i` → `bank_o` (ROW_MAJOR)	2 levels of MUX
`axi_addr_i` → `bank_o` (BANK_INTERLEAVE)	2 levels of MUX
`axi_addr_i` → `bank_o` (XOR_HASH)	5 levels XOR + 2 MUX
`axi_addr_i` → `row_o`	1 level MUX

If XOR_HASH does not meet timing at the target frequency, the FUB can be re-fitted with a single pipeline stage (the seed-XOR can absorb a flop without changing the per-tuple invariant — the seed only changes on quiet points anyway).

8.7 Verification

The RTL decoder and the Python `AddressMapping` class are co-validated by a small cocotb test that picks 10000 random addresses, asks each side to decode, and asserts bit-equality on the (rank, bank, row, col) tuple. This is the primary regression for `addr_mapper_fub`.

8.8 TODO

- Tap matrix for XOR_HASH — the exact bit mapping
- Synthesis hint for the XOR_HASH critical path
- Edge cases for `AW < (ROW_WIDTH + COL_WIDTH + BA + clog2(NR))` (under-decoded address — should it tie-off or error?)

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

9 Read Command CAM (`rd_cmd_cam_fub`)

Module: `rd_cmd_cam_fub.sv` **Location:** `rtl/fub/` **Category:** FUB **Parent:** `ddr2_lpddr2_ctrl` **Status:** Skeleton

---

9.1 Purpose

`rd_cmd_cam_fub` is the **read-side content-addressable storage** of in-flight read commands. It holds the DRAM-layer view (rank, bank, row, col) of every read burst that has been pushed by `axi4_slave_fub` and not yet fully returned on the AXI R-channel.

The CAM is keyed by **AXI ID**, allowing the scheduler to find all reads to a given (rank, bank) without iterating the queue and allowing `rd_data_path_fub` to find the right entry to decrement when a read beat returns.

The read CAM and write CAM are intentionally separate FUBs — see §1.1 for the rationale.

9.2 Parameters

Parameter	Default	Purpose
`RD_CAM_DEPTH`	16	Number of in-flight read slots
`IDW`	`AXI_ID_WIDTH`	Key width
`BURST_LEN_WIDTH`	8	Burst-length field width

9.3 Storage Per Slot

Field	Width	Description
`valid`	1	Slot occupied
`axi_id`	`IDW`	Key — the originating AXI ID
`rank`	`$clog2(NR)`	Decoded rank
`bank`	`BA`	Decoded bank
`row`	`ROW_WIDTH`	Decoded row
`col_start`	`COL_WIDTH`	Starting column
`burst_len`	`BURST_LEN_WIDTH`	Total beats in burst
`beats_returned`	`BURST_LEN_WIDTH`	Beats already returned on R
`issued`	1	Scheduler has picked this entry
`last_beat_at`	small	Cycle of last beat return (for latency telemetry)

Total per-slot width (default config): ~46 bits.

Total storage (16 entries × 46 b): ~92 bytes — small enough to fit comfortably in distributed flops with no SRAM macro.

9.4 Interfaces

9.4.1 Push (from axi4_slave_fub)

Signal	Direction	Width	Description
push_valid	input	1	New read burst arriving
push_ready	output	1	CAM has a free slot
push_id	input	IDW	AXI ID
push_rank	input	$clog2(NR)	from addr_mapper
push_bank	input	BA	
push_row	input	ROW_WIDTH	
push_col	input	COL_WIDTH	
push_len	input	BURST_LEN_WIDTH	

9.4.2 Scheduler Query

The scheduler queries the CAM to: - Find all unissued entries matching a target (rank, bank) — for next-issue selection - Find row-hit candidates (entries whose row == open_row[rank][bank])

Signal	Direction	Description
q_rank_i	input	Query rank
q_bank_i	input	Query bank
q_row_i	input	Query row (for row-hit check)
match_unissued_o	output	One-hot vector of slots matching (rank, bank) and not issued
match_rowhit_o	output	One-hot vector of slots matching (rank, bank, row) and not issued
match_data_o	output	The selected slot's payload (rank, bank, row, col, len)

9.4.3 Issue Notification

When the scheduler picks an entry from this CAM:

Signal	Direction	Description
issued_slot_i	input	Slot index to mark as issued
issued_we_i	input	Mark-issued strobe

9.4.4 Beat Return (from rd_data_path_fub)

Signal	Direction	Description
beat_id_i	input	AXI ID of returning beat
beat_we_i	input	Beat-arrived strobe
entry_complete_o	output	One-hot vector of slots that just completed (last beat of their burst)

On entry-complete the slot's valid is cleared in the next cycle.

9.5 Lookup Mechanism

The CAM uses a **parallel match-line** structure: each slot has a small comparator for {rank, bank} that drives a per-slot match wire. The scheduler reads the full match-vector and picks among matches using a priority encoder (age-priority — oldest slot wins ties).

For the typical RD_CAM_DEPTH of 16 with BA=3, $clog2(NR)=1..2 (1 for single-rank, 2 for quad-rank), this is 16 4–5-bit equality comparators in parallel — trivial silicon.

9.6 Per-ID Beat Counting

Because reads return in-order *per AXI ID* (an AXI ordering rule), the read CAM does not need a per-beat slot table. A slot's beats_returned increments on each beat that returns with that ID, and when beats_returned == burst_len the slot completes and clears.

If AXI_OOO_ACROSS_IDS == false (rare), the SoC has asked for in-order return across all IDs, and the read CAM enforces this by issuing a head-of-queue priority hint to the scheduler — entries are still cleared in arrival order. The OoO-across-IDs default is true; per-ID in-order is the AXI default and is preserved.

9.7 Timing Budget

Path	Budget
push_valid → slot occupy (1 cycle)	1 cycle
q_rank_i / q_bank_i → match_*_o	combinational
beat_id_i →	1 cycle

Path	Budget
`beats_returned[slot]+1`	
`entry_complete_o` → slot clear	1 cycle

The combinational match path is on the scheduler's critical loop and is shared with `wr_cmd_cam_fub` and the bank machines' state matrix. See `ch02_blocks/07_scheduler.md` for the full critical-path breakdown.

9.8 CSR Hooks

- `STATUS.rd_cam_occ` — current occupancy
- `OBS_RD_CAM_HIGH_WATER` — peak occupancy observed since last read

9.9 TODO

- Mermaid for the parallel match-line topology
- Wavedrom: push → scheduler-match → issue → beats return → clear
- Per-ID head-of-queue ordering corner case spec

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

10 Write Command CAM (`wr_cmd_cam_fub`)

Module: `wr_cmd_cam_fub.sv` **Location:** `rtl/fub/` **Category:** FUB **Parent:** `ddr2_lpddr2_ctrl` **Status:** Skeleton

---

10.1 Purpose

`wr_cmd_cam_fub` is the **write-side content-addressable storage** of in-flight write commands. It mirrors the read CAM's role for write traffic but carries different per-slot metadata:

- Pointer into the AXI-side write-data buffer (`w_buf_ptr`)
- Pointer into the AXI-side write-strobe buffer (`strb_ptr`)
- Per-slot beats-issued counter (so the FUB knows when the last W beat has been pushed to DFI write data)

Separation from the read CAM lets the write path sit shallow — writes complete on B response, which is a single AXI event, so the write CAM does not need to track per-beat return.

10.2 Parameters

Parameter	Default	Purpose
WR_CAM_DEPTH	16	Number of in-flight write slots
IDW	AXI_ID_WIDTH	Key width
W_BUF_PTR_WIDTH	$clog2(W_BUF_DEPTH)	Pointer into AXI W buffer
BURST_LEN_WIDTH	8	Burst-length field width

10.3 Storage Per Slot

Field	Width	Description
valid	1	Slot occupied
axi_id	IDW	Key — the originating AXI ID
rank	$clog2(NR)	Decoded rank
bank	BA	Decoded bank
row	ROW_WIDTH	Decoded row
col_start	COL_WIDTH	Starting column
burst_len	BURST_LEN_WIDTH	Total beats
beats_issued	BURST_LEN_WIDTH	Beats already pushed to DFI write data
w_buf_ptr	W_BUF_PTR_WIDTH	Pointer into the AXI W-buffer for next beat
strb_ptr	W_BUF_PTR_WIDTH	Pointer into the per-beat strobe buffer
issued	1	Scheduler has picked this entry
b_pending	1	Awaiting B response from DFI write completion

Total per-slot width: ~58 bits in the default config. Total storage (16 × 58): ~116 bytes, distributed flops.

10.4 Interfaces

10.4.1 Push (from axi4_slave_fub)

Same shape as the read CAM push, plus w_buf_ptr and strb_ptr (which the AXI slave allocates at push time).

10.4.2 Scheduler Query

Same shape as the read CAM. The scheduler matches by (rank, bank) and selects with row-hit priority.

10.4.3 Issue Notification

When the scheduler picks a write, the CAM: 1. Marks issued = 1 2. Begins handing per-beat data to wr_data_path_fub via the beat_data_pull interface (below)

10.4.4 Beat-Pull (to wr_data_path_fub)

Signal	Direction	Description
beat_pull_o	output	Strobe — pull next beat for this slot
beat_pull_slot_o	output	Slot index whose beat is being pulled
beat_pull_ptr_o	output	w_buf_ptr + beats_issued
beat_pull_strb_o	output	strb_ptr + beats_issued
beat_pull_last_o	output	1 when this is the burst's final beat

wr_data_path_fub walks the AXI W buffer at the supplied pointer and pushes to DFI wrdata. On the last beat, it asserts b_pending_set to the CAM.

10.4.5 Completion (from DFI write window expiration)

Signal	Direction	Description
b_pending_set_i	input	Last W beat pushed; await tCWL+tWR window
b_complete_i	input	Write window expired (from xbank_timers)
entry_complete_o	output	Burst is fully complete; signal axi4_slave_fub to push B response

10.5 Difference From Read CAM

Aspect	Read CAM	Write CAM
Lifecycle	Push → issue → beats return → clear	Push → issue → beats issued → write window expires → B →

Aspect	Read CAM	Write CAM
		clear
Beat tracking	Counts beats *returned* from DFI	Counts beats *issued* to DFI; also tracks the post-issue write window
AXI ID echo	`rid` returned with each R beat	`bid` returned once at B-response time
Pointer fields	none (just a beat counter)	`w_buf_ptr`, `strb_ptr` into AXI-side buffers
Completion driver	`rd_data_path` (last beat of R)	`xbank_timers` (write window expiration) → `wr_data_path`

The asymmetry justifies the separate FUBs. Trying to share a CAM would either inflate the read slot to carry write-only fields, or vice versa.

10.6 Timing Budget

Same shape as the read CAM. The combinational match-line path is shared with `rd_cmd_cam_fub` in the scheduler's match aggregation.

10.7 CSR Hooks

- `STATUS.wr_cam_occ` — current occupancy
- `STATUS.wr_b_pending` — count of slots awaiting B response
- `OBS_WR_CAM_HIGH_WATER` — peak occupancy

10.8 TODO

- Mermaid for the issued → beat-pull → b_pending → b_complete lifecycle
- Wavedrom showing a multi-beat write burst from CAM push to B response
- Backpressure path when `wr_data_path` can't keep up with `beat_pull` (rare; DFI wrdata is wider than AXI W)

11 Transaction Queue (txn_queue_fub)

Module: txn_queue_fub.sv **Location:** rtl/fub/ **Category:** FUB **Parent:** ddr2_lpddr2_ctrl
Status: Draft v0.1

> Architectural context: HAS §3.2 txn_queue. This block-level MAS section is the implementation detail: entry format, port arrangement, insert/clear paths, row_hit_cached coherency, backpressure, storage choice.

11.1 Purpose

txn_queue_fub is the **scheduling view** of in-flight DRAM transactions. It is the data structure the scheduler reads every MC clock to pick the next command. The queue is not the source of truth for AXI metadata or beat tracking — that lives in rd_cmd_cam_fub and wr_cmd_cam_fub. The queue carries *only* the fields the scheduler's priority function needs, plus a reverse pointer back to the CAM slot.

The split exists because the scheduler's read path is the hottest in the design: every cycle it touches every entry in parallel. Making the entry narrow (~32 bits, see below) keeps the parallel-comparator fan-in manageable. The CAMs carry the wider per-burst metadata (w_buf_ptr, strb_ptr, beats_returned, etc.) which is only touched on insert, on scheduler pick, and on per-beat completion — never in the per-cycle scheduling loop.

11.2 Relationship to the CAMs

Each entry in txn_queue reflects exactly one slot in rd_cmd_cam or wr_cmd_cam. The cam_slot_idx field is the reverse pointer:

```
txn_queue[q].cam_slot_idx ⟶ (is_write ? wr_cmd_cam[cam_slot_idx] :
rd_cmd_cam[cam_slot_idx])
```

When the scheduler picks txn_queue[q], it issues an "mark issued" strobe to the corresponding CAM using cam_slot_idx (see scheduler_fub, §2.7). When the CAM signals entry-complete (read: last beat returned; write: B-channel quiet point reached), it strobes back to clear txn_queue[q].

The queue and CAMs are sized independently:

Structure	Default depth	Reason
txn_queue	16	Scheduling pool — depth bounded by parallel-

Structure	Default depth	Reason
		comparator cost
`rd_cmd_cam`	16	One slot per pending read
`wr_cmd_cam`	16	One slot per pending write

In the default 16+16+16 configuration, the queue is the bottleneck — at most 16 transactions can be queued for scheduling at any time, regardless of how many slots are free in each CAM individually. The CAMs are not allowed to oversubscribe the queue (the `axi4_slave_fub` push path stalls on `txn_queue` full, not on CAM-full). The two CAMs *can* be sized larger than the queue if the read-path and write-path have very different burst-completion latencies — but that's not v1.

11.3 Synthesis Parameters

Parameter	Source	Effect
`TXN_QUEUE_DEPTH`	top (default 16)	Number of parallel-readable entries
`AGE_MAX`	top (default 256)	Age counter saturation; per-entry `clog2(AGE_MAX) = 8` bits
`ROW_WIDTH`	top	Width of the `row` field
`NUM_RANKS`	top	Width of the `rank` field
`NUM_BANKS`	top	Width of the `bank` field

11.4 Entry Format

Each slot carries the **scheduling view** — the minimum metadata the scheduler's priority function needs.

Field	Width	Description
`valid`	1	Slot occupied
`state`	2	`{FREE, PENDING, ISSUED, COMPLETING}` (see lifecycle below)
`is_write`	1	Picks rd vs wr CAM for the reverse pointer
`rank`	`$clog2(NUM_RANKS)`	Target rank (1-2 bits)
`bank`	`$clog2(NUM_BANKS)`	Target bank (3 bits for NB=8)

Field	Width	Description
row	ROW_WIDTH	Target row (14 bits default)
row_hit_cached	1	Cached at insert; refreshed on bank-state change
age	$clog2(AGE_MAX)	Saturating age counter (8 bits default)
cam_slot_idx	$clog2(CAM_DEPTH)	Reverse pointer to rd_cmd_cam or wr_cmd_cam slot (4 bits)
Total per slot	**~32 bits**	(default: NR=1, NB=8, ROW=14, AGE_MAX=256, CAM=16)

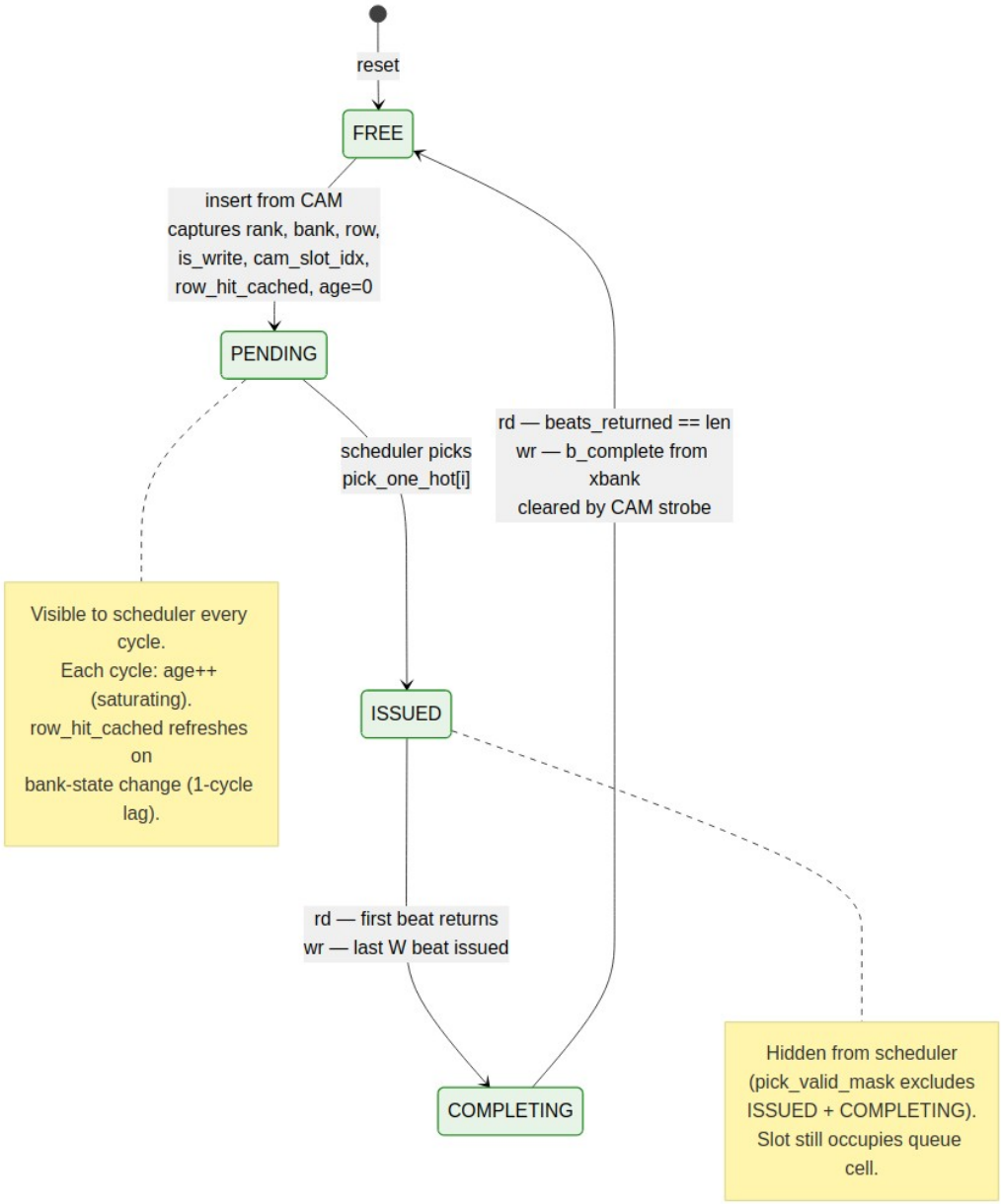
Total queue storage at depth 16: **~64 bytes** (16 × 32 b). This is distributed flops — no SRAM macro. Even at depth 64 it's 256 bytes, still trivially distributed.

Fields *not* in the queue entry (these stay in the CAM):

- col_start, burst_len — only consulted at issue time (Stage 4 of scheduler), not every cycle
- axi_id — opaque to the scheduler; only the CAM and axi4_slave_fub need it
- w_buf_ptr, strb_ptr, beats_returned, beats_issued — beat-level tracking, not scheduling

At issue time, the scheduler reads col_start and burst_len via the cam_slot_idx reverse pointer — a single read from the CAM's lookup port. This is one cycle slower than carrying them in the queue, but it keeps the queue narrow.

11.5 Entry Lifecycle



txn_queue Entry State Diagram

Source: 06_txn_queue_entry_state.mmd

Four states tracked per slot:

State	Visible to scheduler?	Counts toward queue depth?	Notes
FREE	No	No	Empty slot, available for insert
PENDING	Yes (eligible)	Yes	Scheduler is searching for this entry
ISSUED	No (masked out)	Yes	Picked by scheduler; column command in flight
COMPLETING	No (masked out)	Yes	Last beat / write-window draining

The scheduler's pick_valid_mask (see §2.7 Stage 2) explicitly excludes ISSUED and COMPLETING so the same entry can never be picked twice. The mask is (state == PENDING).

ISSUED → COMPLETING is the same edge in both rd and wr cases but triggered by different signals:

- **Read**: first R beat returns from DFI → rd_data_path_fub strobes entry_transition_completing via the CAM.
- **Write**: last W beat is pushed to DFI write data → wr_data_path_fub strobes the same edge.

The COMPLETING → FREE edge is driven by the CAM's entry_complete_o strobe (see §2.4 and §2.5). The queue does not own completion-detection logic; it follows the CAM.

11.6 Ports

11.6.1 Insert (from CAMs)

Signal	Direction	Width	Description
ins_valid_i	input	1	New entry to push
ins_ready_o	output	1	Queue has a free slot
ins_is_write_i	input	1	
ins_rank_i	input	$clog2(NR)	from addr_mapper
ins_bank_i	input	$clog2(NB)	
ins_row_i	input	ROW_WIDTH	
ins_cam_slot_i	input	$clog2(CAM_DEPTH)	Reverse pointer

row_hit_cached is **not** an insert port — it's computed inside this FUB at insert time by querying bank_state_i[ins_rank][ins_bank].open_row (see "Insert Path" below). This keeps the insert-time row-hit computation local to the queue.

11.6.2 Parallel Read (to scheduler — every cycle)

Signal	Direction	Width	Description
q_entries_o[TXN_QUEUE_DEPTH-1:0]	output	per entry-format table	Live snapshot of all slots, parallel

This is a wide bus — at the default TXN_QUEUE_DEPTH = 16 and ~32 bits per entry, it's a 512-bit bus. The scheduler reads it combinationally; there's no read-port arbitration because there's only one reader.

11.6.3 Bank-State Watchpoint (from bank machines)

Signal	Direction	Width	Description
bank_state_i[NR][NB]	input	per bank_machine	Live bank state (used at insert for row_hit_cached)
bank_open_row_changed_i[NR][NB]	input	NR×NB	One-cycle strobe when a bank's open row changes
bank_new_open_row_i[NR][NB]	input	NR×NB × ROW_WIDTH	New open-row value, valid the cycle after the strobe

The _changed + _new_open_row pair is what drives the per-entry row_hit_cached refresh path (see "Row-Hit Cache Coherency" below).

11.6.4 Scheduler Pick (from scheduler)

Signal	Direction	Width	Description
pick_one_hot_i	input	TXN_QUEUE_DEPTH	One-hot vector of which slot the scheduler picked
pick_strobe_i	input	1	Pick happened this cycle

On pick_strobe_i, the indicated slot transitions PENDING → ISSUED.

11.6.5 CAM Completion (from CAMs)

Signal	Direction	Width	Description
entry_transition_i	input	2	01 = ISSUED → COMPLETING, 10 = COMPLETING → FREE
entry_transition_slot_i	input	$clog2(TXN_QUEUE_DEPTH)	Slot index that's transitioning
entry_transition_strobe_i	input	1	Transition strobe

11.6.6 Backpressure Output

Signal	Direction	Width	Description
q_high_water_o	output	1	High when occupancy ≥ SCHED_TUNING.txn_queue_high_water
q_full_o	output	1	All slots are non-FREE
q_occupancy_o	output	$clog2(DEPTH+1)	Current occupancy (number of slots not in FREE state)

q_high_water_o drives the AXI backpressure path in axi4_slave_fub — awready / arready deasserts when the queue is near full. q_full_o is the hard backpressure (no inserts accepted).

11.7 Insert Path

When axi4_slave_fub has decoded a new burst and pushed it to either CAM, the CAM in turn pushes to this queue:

```
cycle T:   CAM push from axi4_slave  →  ins_valid asserted, ins_ready
asserted
cycle T:   queue selects free_slot via priority encoder over (state ==
FREE)
cycle T:   row_hit_cached_init = (bank_state[ins_rank][ins_bank].state
== ACTIVE
                                  AND bank_state[ins_rank]
[ins_bank].open_row == ins_row)
cycle T+1: free_slot.{is_write, rank, bank, row, row_hit_cached,
```

```
cam_slot_idx} latched
cycle T+1: free_slot.state = PENDING, age = 0
```

The insert is a **two-cycle latency** from CAM push to first scheduler visibility, but the queue can accept one new insert per cycle (the comparator network for `free_slot` finds the next available slot combinationally; the latch is registered).

`row_hit_cached_init` is the only nontrivial computation at insert time. It's a single comparison: does the target bank's open row match the inserted row? If the bank is IDLE, it's automatically 0 (no row is open). If the bank is in the middle of a row change (ACTIVATING or PRECHARGING), it's also 0 — the cached value will be refreshed in the next cycle when the bank settles via the watchpoint path below.

11.8 Row-Hit Cache Coherency

The `row_hit_cached` field is computed at insert (above) and **refreshed** any time a bank's open row changes.

The refresh path:

```
cycle T:   bank_machine[r,b] transitions ACTIVE → ACTIVE-on-new-row
           (via PRE then ACT)  OR  IDLE → ACTIVE (via ACT)
cycle T:   bank_machine[r,b] strobes bank_open_row_changed[r][b] = 1
           and presents bank_new_open_row[r][b] = R_new
cycle T+1: txn_queue refresh path:
           for each slot i:
             if entries[i].state in {PENDING}
                AND entries[i].rank == r AND entries[i].bank == b:
               entries[i].row_hit_cached = (entries[i].row == R_new)
```

The refresh is broadcast-parallel: all `TXN_QUEUE_DEPTH` entries are checked in parallel (their `(rank, bank)` matches are computed; the matching set has its `row_hit_cached` recomputed). The refresh fires for `ISSUED` and `COMPLETING` entries too, but those are masked out by the scheduler anyway — refreshing them is harmless and cheaper than gating.

Why a one-cycle lag is acceptable. In the cycle the bank state changes, the scheduler may still observe stale `row_hit_cached`. The worst outcome is the scheduler picks an entry as a "row hit" that isn't, or vice versa. The bank_machine catches this — `accepts_rd` / `accepts_wr` are gated on the current open_row, so the scheduler can't issue an *incorrect* column command. It would issue a bare RD/WR when an RDA/WRA was warranted (or vice versa), costing at most one extra PRE later. This is the trade documented in scheduler §2.7.

The synchronous broadcast (rather than asynchronous combinational re-eval) keeps the scheduler's Stage-1 path off the bank_machine state machines. If we tried to make this combinational, the Stage-1 critical path would route through every bank machine's open_row

register, every queue entry's row comparator, and every priority mask — easily 8-10 LUT levels, ruining the timing budget.

11.9 Age Counter Saturation

Every cycle, every `PENDING` entry's `age` increments by 1. The counter saturates at `AGE_MAX - 1`. Concretely with `AGE_MAX = 256`:

- New entry: age = 0
- After 1 cycle: age = 1
- After 255 cycles: age = 255
- After 256+ cycles: age = 255 (saturated)

The scheduler's age priority encoder treats saturated entries equally — once an entry has been waiting "long enough," it has maximum priority and ties break by slot index. In normal traffic, ages rarely saturate; in pathological cases (a row-conflict victim behind a stream of row-hits) saturation provides the anti-starvation guarantee.

The runtime override `SCHED_TUNING.age_max_runtime` clips the effective saturation point lower than the build-time `AGE_MAX` — useful for characterization sweeps to shorten the saturation timescale without rebuilding.

Increment gating. Age does *not* increment when `state` is `ISSUED` or `COMPLETING`. Those entries are out of the scheduling pool; aging them further would skew the priority for any subsequent entries that get re-queued (which doesn't currently happen in v1, but the gate is cheap insurance).

11.10 Storage Choice

At the default `TXN_QUEUE_DEPTH = 16`, total storage is ~64 bytes. Distributed flops are the right answer — an SRAM macro would have higher access latency and the macro overhead is significant for so little data.

The crossover point where SRAM becomes attractive is around `TXN_QUEUE_DEPTH ≥ 64` (256 bytes), and even then only if the scheduler can be re-architected to issue from a partial snapshot rather than the full queue. Currently, the scheduler reads all entries in parallel — SRAM ports can't deliver that. So in v1 we cap at distributed flops; the parameter range is `4 … 64`.

For depth = 64, the entry bus is 64 × 32 = 2048 bits — wide but routable on 7-series. Stage-1 priority comparators scale linearly with depth (16 → 64 is 4× the comparators), and the age priority encoder tournament grows by $\log_2(4) = 2$ levels. The critical-path budget in §2.7 absorbs this at the 200 MHz target; at 500 MHz the scheduler will need flop insertion as noted.

11.11 CSR Hooks

CSR field	Source signal	Use case
STATUS.txn_queue_occ (R)	q_occupancy_o	Live occupancy
OBS_TXN_QUEUE_DEPTH_MAX (R)	High-water-mark register	Peak occupancy since last read (HAS §6.3)
OBS_TXN_QUEUE_DEPTH_AVG (R)	Rolling time-average	Time-averaged occupancy
STATUS.txn_queue_full_pulses (R)	Counter — number of times q_full_o asserted	Hard-backpressure event counter
SCHED_TUNING.txn_queue_high_water (R/W)	Threshold for q_high_water_o	Software-tunable soft-backpressure point

The _high_water threshold defaults to TXN_QUEUE_DEPTH - 2 so AXI has a few-burst headroom before hard backpressure. Software can tune it lower for tighter latency control.

11.12 Verification Notes (cocotb test plan)

Scenario	What it proves
Single insert, single pick, single clear	Smoke: FREE → PENDING → ISSUED → COMPLETING → FREE
Insert at full speed (1 per cycle) until full	Insert throughput; q_full_o asserts on the 17th
Bank-state change while N entries target that (rank, bank)	Broadcast row-hit refresh works on all matching slots
Bank-state change *during* a pick on the same slot	Pick is honored; refresh applies to next cycle's view
Age saturation across all queue slots	All saturate at AGE_MAX - 1; tie-break by slot index
age_max_runtime = 16, age saturates earlier	Runtime clip works

Scenario	What it proves
than `AGE_MAX`	
Insert + pick + complete in the same cycle (different slots)	The four ports are non-conflicting
`txn_queue_high_water = 8`; AXI backpressure asserts at occupancy 8	Soft backpressure threshold
`q_full_o` blocks CAM push; CAM stalls AXI	Hard backpressure path
Insert with `is_write = 0` and `is_write = 1` interleaved	Reverse-pointer routing to correct CAM
`entry_transition` with `slot` out-of-range (illegal)	Assertion fires; entry untouched
Reset during mid-life: all slots clear, no spurious completions emitted	Reset behavior

11.13 Open Questions / Future Work

- Should the queue allow **re-queue** of a `COMPLETING` entry that fails (e.g., a DFI rddata timeout)? Currently the CAM owns failure-recovery; if it decides to retry, it would have to push a new queue entry. A "re-queue without losing age" path would be cheaper but adds complexity. Defer to v2.
- Should `cam_slot_idx` be **wider** than `$clog2(CAM_DEPTH)` to allow over-sized CAMs? Currently sized to match the smaller of the two CAM depths. If the rd-CAM grows to 32 and wr-CAM stays at 16, the field width adapts at elaboration; this is fine but the field is technically over-provisioned for write entries. Not worth fixing in v1.
- The broadcast `row_hit_cached` refresh fires for `ISSUED` / `COMPLETING` entries too. These are masked at the scheduler anyway. The gate would save a few comparator switches but no flops — drop the gate optimization. Document the decision and move on.

12 FR-FCFS Scheduler (scheduler_fub)

Module: scheduler_fub.sv **Location:** rtl/fub/ **Category:** FUB **Parent:** ddr2_lpddr2_ctrl
Status: Draft v0.1

> Architectural context: HAS §3.2 (scheduler, priority function, lookahead, force_inorder, refresh-priority). This block-level MAS section is the implementation view: signals, internal datapath, pipeline staging, critical paths, and verification scenarios.

12.1 Purpose

scheduler_fub is the central command issue engine. Every MC clock cycle it:

1. Snapshots txn_queue and all bank_machine states
2. Computes a per-entry readiness vector (which queue entries *could* issue right now)
3. Applies priority masking (init, refresh, force-inorder, row-hit-first, age tie-break)
4. Picks at most one winner via parallel priority encoders
5. Generates the DRAM command (issue_op, operands) and the CAM "mark issued" strobe

Issue rate is **one command per MC clock**. Multi-cycle commands (LPDDR2's 2-cycle CA encodings) are absorbed inside cmd_encoder_fub and gear_dfi_fub; the scheduler still sees one issue per cycle.

The block is the **single hardest synthesis-timing FUB** in the design — the parallel match-line into per-entry readiness, then into priority encoders, is what sets the MC clock frequency ceiling. Everything else has slack.

12.2 Synthesis Parameters

Parameter	Source	Effect on this FUB
SCHEDULER_MODE	top	"000" synthesizes the full FR-FCFS comparator network; "INORDER" synthesizes only the first-ready FIFO path (smaller area, no row-hit reordering).

Parameter	Source	Effect on this FUB
TXN_QUEUE_DEPTH	top (default 16)	Number of parallel readiness comparators
LOOKAHEAD_DEPTH_MAX	top (default 4)	Width of the lookahead window peek into txn_queue
PAGE_POLICY	top	Picks which Stage-4 auto-precharge decoder is synthesized
NUM_RANKS, NUM_BANKS	top	Width of the bank-state input matrix
AGE_MAX	top (default 256)	Width of per-entry age counter (clog2(AGE_MAX))

The "INORDER" synthesis variant is a different decoder tree, not a runtime gate of the OoO tree. The runtime SCHED_TUNING.force_inorder bit is honored *only* when SCHEDULER_MODE == "OOO" was synthesized (the bit becomes a tied observation in INORDER builds, per HAS §3.2).

12.3 Interface

12.3.1 Inputs

Signal	Direction	Width	Description
mc_clk, mc_rst_n	input	1, 1	MC clock + async-assert / sync-deassert reset
q_entries_i[TXN_QUEUE_DEPTH-1:0]	input	per HAS §3.2	Live queue snapshot: per entry {valid, is_write, rank, bank, row, col, burst_len, row_hit_cached, age, state, cam_slot_idx}
bank_state_i[NUM_RANKS][NUM_BANKS]	input	per bank_machine_fub	Per-(rank, bank): state, open_row, accepts_act/rd/wr/pre/ref
xbank_blocks_i	input	struct	blocks_act_per_rank[NR], blocks_xrank_rd, blocks_xrank_wr
refresh_wants_i	input	1	One-bit “refresh due now” from refresh_mgr_fub

Signal	Direction	Width	Description
refresh_done_for_i[NUM_RANKS][NUM_BANKS]	input	NR×NB	Per-(rank, bank) “refresh handshake granted” from refresh_mgr_fub
init_in_progress_i	input	1	From init_engine_fub — blocks all normal traffic
predict_hit_i[NUM_RANKS][NUM_BANKS]	input	NR×NB	HAPPY predictor output (tied 0 when PAGE_POLICY != HAPPY_HYBRID)
cfg_force_inorder_i	input	1	SCHED_TUNING.force_inorder runtime override
cfg_lookahead_active_i	input	4	SCHED_TUNING.lookahead_active
cfg_page_policy_or_i	input	2	REFRESH_TUNING.page_policy_or
cfg_happy_enable_i	input	1	SCHED_TUNING.happy_enable

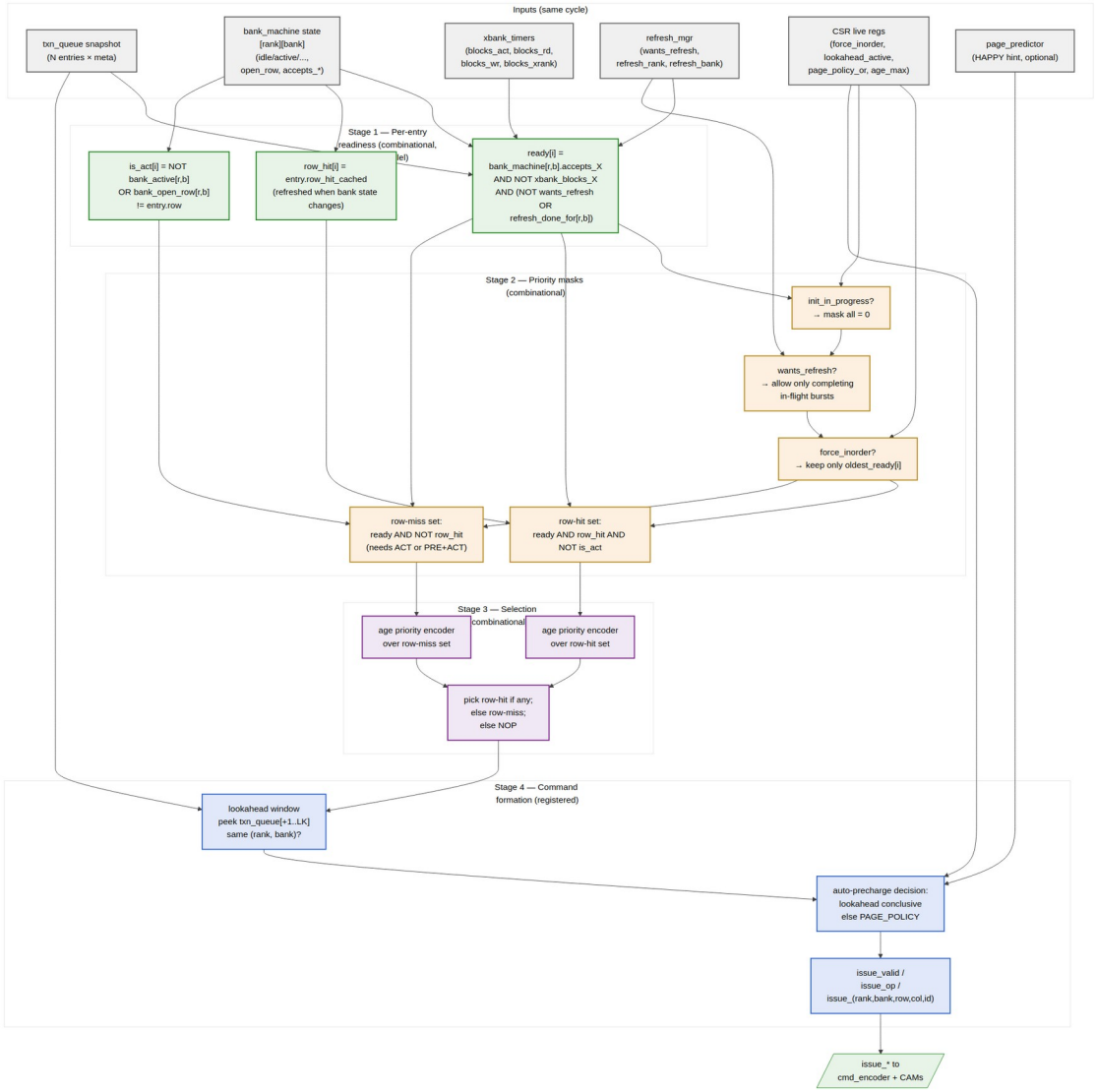
12.3.2 Outputs

Signal	Direction	Width	Description
issue_valid_o	output	1	A command is being issued this cycle
issue_op_o	output	4	One of {NOP, ACT, RD, RDA, WR, WRA, PRE, PREA, REF, REFPB, MRS, ZQCS, ZQCL}
issue_rank_o	output	$clog2(NR)	Target rank
issue_bank_o	output	$clog2(NB)	Target bank
issue_row_o	output	ROW_WIDTH	Target row (for ACT) or open-row pass-through (for RD/WR)
issue_col_o	output	COL_WIDTH	Target column (for RD/WR)

Signal	Direction	Width	Description
issue_axi_id_o	output	AXI_ID_WIDTH	Originating AXI ID (passed to the CAM for completion routing)
issue_cam_slot_o	output	clog2(CAM_DEPTH)	Index into rd_cmd_cam or wr_cmd_cam to mark “issued”
issue_is_write_o	output	1	Picks wr_cmd_cam vs rd_cmd_cam for the mark-issued strobe
predict_query_o	output	struct	(rank, bank, row) query to page_predictor_fub for stage-4 auto-pre
dbg_inorder_active_o	output	1	(debug) High when current cycle picked the in-order path
dbg_refresh_blocking_o	output	1	(debug) High when wants_refresh reduced the candidate pool

12.4 Decision Pipeline

The decision is a single MC-clock-cycle combinational pipeline broken into four conceptual stages, finishing in a registered output. Stages 1–3 are combinational; Stage 4 latches the final command into output flops.



Scheduler Priority Pipeline

Source: 05_scheduler_priority_pipeline.mmd

12.4.1 Stage 1 — Per-Entry Readiness (combinational, N parallel comparators)

For each of `TXN_QUEUE_DEPTH` entries, in parallel:

```
ready[i] = q_entries[i].valid
         AND  (q_entries[i].state == PENDING)
         AND  bank_machine[r,b].accepts_X
         AND  NOT xbank_blocks_X
         AND  ( NOT refresh_wants
                OR  bank_machine[r,b].in_refresh_done_set )
```

```
is_row_hit[i] = q_entries[i].row_hit_cached
                ( cached at queue insert; refreshed when
bank_machine[r,b]
                  state changes — see "Cache Coherency" below )

needs_act[i]  = NOT bank_active[r,b]
                OR  bank_open_row[r,b] != q_entries[i].row
```

Where X is `rd` if `is_write == 0`, `wr` if `is_write == 1`. `r = q_entries[i].rank`, `b = q_entries[i].bank`.

`ready[i]` answers "could I issue *some* command for this entry this cycle without violating any per-bank or cross-bank timer?" `is_row_hit[i]` answers "if I do, is it a column command on the already-open row?" `needs_act[i]` answers "is the preliminary command actually ACT (or PRE+ACT) rather than RD/WR?"

12.4.2 Stage 2 — Priority Masking (combinational)

Five masks, computed in parallel from the Stage-1 vector and the config inputs:

Mask	Definition	Effect
`init_mask`	`init_in_progress ? 0 : ready[i]`	Block all normal entries during init
`refresh_mask`	`refresh_wants ? (ready[i] AND bank_in_refresh_done_set) : ready[i]`	Allow only entries whose target rank+bank has already been granted the refresh handshake (i.e., already transitioned out of REFRESHING, or never entered it)
`inorder_mask`	`force_inorder ? oldest_ready_one_hot : ready[i]`	Collapse to first-ready FIFO
`row_hit_mask`	`ready[i] AND is_row_hit[i] AND NOT needs_act[i]`	Row-hit candidates (column-only)
`row_miss_mask`	`ready[i] AND (NOT is_row_hit[i] OR needs_act[i])`	Row-miss candidates (need ACT or PRE+ACT)

The composed candidate vectors are:

```
cands_row_hit  = init_mask AND refresh_mask AND inorder_mask AND
row_hit_mask
cands_row_miss = init_mask AND refresh_mask AND inorder_mask AND
row_miss_mask
```

12.4.3 Stage 3 — Selection (combinational priority encoders)

Two parallel age-priority encoders, then a 2:1 mux:

```
winner_hit  = age_pe(cands_row_hit)    // index of oldest entry in
cands_row_hit, or none
winner_miss = age_pe(cands_row_miss)   // index of oldest entry in
cands_row_miss, or none

picked = winner_hit  IS_VALID ? winner_hit
       : winner_miss IS_VALID ? winner_miss
                              : NONE   // issue NOP
```

The age priority encoder is the slowest path in this stage — it's an `N`-entry tournament on `clog2(AGE_MAX) = 8`-bit ages. For `N = 16` (the default), that's 4 levels of pairwise compare-and-select. Synthesizable in two LUT layers per level on 7-series; ~30 LUT-delays in the typical config.

12.4.4 Stage 4 — Command Formation (registered)

Once an entry is picked, Stage 4 turns it into a DRAM command. This stage **may pre-flop intermediate signals** for timing closure on the highest clock targets (see "Pipeline Staging" below).

1. **Lookahead peek** — examine the next `cfg_lookahead_active` queue entries *after* the picked one to see if any target the same (rank, bank).
2. **Auto-precharge decision** — per HAS §3.2 (lookahead first, fallback to `PAGE_POLICY`):
   - Same-bank entry found AND its row matches the open row → **keep open** → bare RD / WR
   - Same-bank entry found AND its row differs → **close** → RDA / WRA
   - No same-bank entry within window → consult `cfg_page_policy_or`:
     - OPEN → bare RD / WR
     - CLOSE → RDA / WRA
     - HAPPY_HYBRID → query `predict_hit_i[r][b]`; if predicted hit → bare; else RDA/WRA
3. **Op encoding** — generate the 4-bit `issue_op_o` from `(is_write, needs_act, auto_pre)`:
   - `needs_act` and bank state == IDLE → `ACT`
   - `needs_act` and bank state == ACTIVE (different row) → `PRE` (the column command follows next cycle after tRP)
   - `NOT needs_act` and `is_write` → `WR` or `WRA`
   - `NOT needs_act` and `NOT is_write` → `RD` or `RDA`

4. **Latch** outputs into the issue flops.

12.4.5 Refresh Window Behavior

When `refresh_wants_i` is high, `refresh_mgr_fub` is asserting `refresh_req` to one or more bank machines. Those bank machines drive `refresh_done_for_i[r][b] = 1` only after reaching IDLE and handing over the grant. While `refresh_done_for_i[r][b] == 0`, that bank's entries are masked out — no new column commands to a bank that's about to refresh.

Bank machines for *other* ranks (in the multi-rank REFab round-robin) or *other* banks (in REFpb) remain available, so the scheduler continues to drain those during the refresh window. This is what makes per-rank REFab dispatch beneficial — the non-refreshing ranks keep their throughput.

When all required refreshes complete and `refresh_wants_i` drops, the masks lift and normal FR-FCFS resumes the next cycle.

12.5 Cache Coherency: `row_hit_cached`

`q_entries[i].row_hit_cached` is computed at queue insertion (by `axi4_slave_fub` via a one-cycle query to `bank_machine[r,b].open_row`) and stored in the queue entry. This avoids re-querying every cycle. But the bank state can change between insertion and selection:

- When `bank_machine[r,b]` transitions ACTIVE → ACTIVE on a different row (via PRE + ACT to a new row), all `q_entries` with `(rank, bank) == (r, b)` need their `row_hit_cached` recomputed against the new `open_row`.
- When `bank_machine[r,b]` transitions IDLE → ACTIVATING, the queue entries get `row_hit_cached = 1` for entries whose row matches the in-flight ACT.

This is implemented as a **broadcast update**: on every bank state transition that affects the open row, the bank machine emits `open_row_changed[r][b]` + the new `open_row`. The queue has a per-entry update path that, in parallel, recomputes `row_hit_cached`. This is a one-cycle update — the recomputed value is visible to the scheduler in the cycle *after* the bank transition.

The one-cycle lag means the scheduler may make an "outdated row-hit" decision once per row change. The downside is one mis-categorized RD/WR (issued as bare when RDA was warranted, or vice versa); the worst case is one extra PRE later. This is a deliberate trade — the alternative is a combinational broadcast from bank state through every queue entry's row-hit comparator, which would inflate the Stage-1 critical path.

12.6 Pipeline Staging

The default build is a **single-cycle combinational** scheduler — Stages 1–3 are combinational; Stage 4's auto-precharge decoder is combinational; only the final issue-flop is registered. On 7-series FPGA at 200 MHz this closes timing comfortably. For 14 nm targets at 500+ MHz, two timing-closure escape hatches:

1. **Stage 3 → Stage 4 flop insertion** — register `picked` between stages. Adds one cycle of issue latency. Throughput is unchanged because the pipeline is single-issue anyway; the only visible difference is that a request inserted at cycle T can first issue at cycle T+3 instead of T+2.

2. **Stage 1 partition by bank** — partition the readiness comparator network by (rank, bank) so each partition produces a per-bank ready vector independently, then aggregate. Useful when `NUM_RANKS × NUM_BANKS` is small (the default 1×8 = 8 partitions) and routing congestion dominates.

These are not on by default; they're synthesis-script switches, not RTL parameters.

12.7 Strict In-Order Mode (collapse)

When `cfg_force_inorder_i = 1` (and `SCHEDULER_MODE == "OOO"` is synthesized):

- `inorder_mask` collapses to `oldest_ready_one_hot` — a one-hot vector that picks the *oldest valid ready entry* by age.
- This drops the row-hit prioritization (no row-hit vs row-miss split — both go through `cands_row_miss` since age order rules).
- Refresh priority is preserved (JEDEC requirement).
- Lookahead and HAPPY remain on at elaboration but can be independently disabled at runtime (`cfg_lookahead_active = 0`, `cfg_happy_enable = 0`) for a pure first-ready FIFO.

When `SCHEDULER_MODE == "INORDER"` is synthesized:

- The Stage-2 / Stage-3 priority encoders are different RTL: the row-hit/row-miss split is omitted; only the oldest-ready selector is built.
- `cfg_force_inorder_i` becomes a tied observation bit at `STATUS.force_inorder_obs = 1`.

12.8 Critical-Path Analysis (rough budget at 500 MHz target, 2 ns cycle)

Path	Levels (approx)	Budget
`bank_state_i.accepts_X` → `ready[i]` → row-hit mask → `cands_row_hit[i]`	4 LUT levels	0.7 ns
`cands_row_hit` → age priority encoder → `winner_hit` (16-entry tournament)	4 levels × 2 LUT each = 8	1.2 ns
`winner_hit / winner_miss` → 2:1 mux → `picked`	1 LUT level	0.2 ns
Routing / setup margin		0.3 ns
Total Stage 1–3		**2.4 ns**

So at 500 MHz the path **misses by ~400 ps** and needs Stage 3 → Stage 4 flop insertion (the first escape hatch above). At 200 MHz target (5 ns cycle, embedded SoC default), the path closes with ~2.6 ns of slack — no escape hatch needed.

The 500 MHz analysis is FPGA-pessimistic; ASIC targets close 500 MHz on this path comfortably with standard pipelining.

12.9 CSR Hooks

The scheduler drives several CSR observability fields:

CSR field	Source signal	Use case
`STATUS.issue_op_obs` (R)	Last issued `issue_op_o`	Bring-up debug: what was the last DRAM command
`STATUS.scheduler_idle_pct` (R)	Rolling counter — fraction of cycles `issue_valid_o = 0`	Bandwidth-headroom telemetry
`OBS_ROW_HIT_RANK<R>_BANK<N>` (R)	Incremented when row-hit path wins for that bank	Per-bank row-hit rate (HAS §6.3)
`OBS_INORDER_FALLBACKS` (R)	Incremented when `force_inorder` ate a row-hit	Tells software how much OoO is currently giving up

CSR field	Source signal	Use case
	win	
`OBS_LOOKAHEAD_HITS` (R)	Incremented when Stage-4 lookahead was conclusive	Tunes `LOOKAHEAD_DEPTH_MAX` characterization sweep
`STATUS.force_inorder_obs` (R)	Echo of effective force-inorder state	Software-readable confirmation

12.10 Verification Notes (cocotb test plan)

Scenario	What it proves
Single-write, single-bank, single-rank	Smoke test: ACT → WR → b_complete cycle
Single-read, single-bank, single-rank	Smoke test: ACT → RD → beat return
Two reads, same bank, same row, oldest first	Row-hit reordering not invoked (already in order)
Two reads, same bank, *different* rows, oldest is row-conflict	OoO: younger row-hit wins over older row-miss
Same as above with `force_inorder = 1`	OoO disabled: older row-miss wins despite younger row-hit
Cross-bank: rd to bank 0 and rd to bank 1, bank 0 row-miss, bank 1 row-hit	Bank parallelism: bank-1 issue fires first
Cross-rank (NUM_RANKS=2): rd to rank 0 bank 3 and rd to rank 1 bank 3	Per-rank bank parallelism
Refresh during in-flight burst	Refresh window correctly drains and resumes
Refresh during multi-rank traffic (REFab on rank 0, traffic on rank 1)	Per-rank REFab dispatch: rank-1 traffic continues
Age-saturation under sustained row-conflict from younger IDs	Older row-conflict eventually wins via age priority
`LOOKAHEAD_DEPTH_MAX = 4`, lookahead concludes auto-precharge correctly	Lookahead path
`LOOKAHEAD_DEPTH_MAX = 0`, HAPPY	Fallback path through page

Scenario	What it proves
predictor used	predictor
`PAGE_POLICY = CLOSE`, every column command is auto-pre	CLOSE policy
Init-in-progress blocks normal traffic	`init_mask` gating
Queue full + back-pressure to AXI	`axi4_slave` does not accept new bursts

12.11 Open Questions / Future Work

- Should QoS (`awqos` / `arqos`) be promoted from the v1 "no behavior" pass-through to a real priority boost in the FR-FCFS function? Currently HAS §3.2 leaves QoS as a side-band hint for v2. The hook would be a `qos_boost_mask` between the row-hit and row-miss stages.
- The cross-rank read-to-write turnaround (`tRTRS` + write window) lives in `xbank_timers_fub` today; the scheduler consumes the gate but does not pre-plan around it. A smarter scheduler could **batch by rank** (issue all ready reads on rank 0, then all on rank 1) to amortize `tRTRS`. This is a v2 feature; v1 takes the simpler per-cycle gating.
- Whether `OBS_INORDER_FALLBACKS` is useful for characterization is not yet validated; the counter is cheap, but if it's never read by the bring-up team it can be dropped in v2.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

13 HAPPY Page Predictor (`page_predictor_fub`)

Module: `page_predictor_fub.sv` **Location:** `rtl/fub/` **Category:** FUB (conditional — synthesized only when `PAGE_POLICY == HAPPY_HYBRID`) **Parent:** `ddr2_lpddr2_ctrl` **Status:** Draft v0.1

> Architectural context: HAS §3.2 `page_predictor`. The algorithm view (query + update flow) is in `ddr2_lpddr2_has/assets/mermaid/09_happy_predictor.png` — refer to

it for the algorithm. This block-level MAS section is the implementation view: hash inputs, table organization, multi-rank handling, warmup / hysteresis, storage choice, scheduler timing.

13.1 Purpose

`page_predictor_fub` is the HAPPY hybrid page-conflict predictor (Ghasempour et al., 2015). It answers one question for the scheduler:

> "For a request I am about to issue to (rank, bank), will the *next* access to this (rank, bank) hit the row I am about to leave open, or conflict with it?"

The answer drives the Stage-4 auto-precharge fallback in `scheduler_fub` (§2.7) when the lookahead window is inconclusive. A "next will hit" prediction → issue bare RD/WR (keep row open). A "next will conflict" prediction → issue RDA/WRA (close row in flight to save tRP later).

The FUB is **synthesized only when `PAGE_POLICY == HAPPY_HYBRID`** at elaboration. Builds with `PAGE_POLICY == OPEN` or `PAGE_POLICY == CLOSE` do not instantiate this FUB; the scheduler ties its `predict_hit_i` input to 1 (default-to-open) or 0 (default-to-close) respectively.

A runtime kill-switch (`SCHED_TUNING.happy_enable`) lets software A/B-test the predictor's contribution without rebuilding. When `happy_enable == 0`, the FUB stays in silicon but its output forces `predict_hit = 1` (default open).

13.2 Conditional Instantiation

```
generate
    if (PAGE_POLICY == "HAPPY_HYBRID") begin : g_happy
        page_predictor_fub #(
            .PAGE_PREDICTOR_TABLE_BITS (PAGE_PREDICTOR_TABLE_BITS),
            .NUM_RANKS                 (NUM_RANKS),
            .NUM_BANKS                 (NUM_BANKS),
            .ROW_WIDTH                 (ROW_WIDTH)
        ) u_page_predictor ( ... );

        assign predict_hit_to_sched = u_page_predictor.predict_hit_o;
    end
    else if (PAGE_POLICY == "OPEN") begin : g_open
        assign predict_hit_to_sched = '1;        // always predict
"hit" → bare RD/WR
    end
    else begin : g_close // CLOSE
```

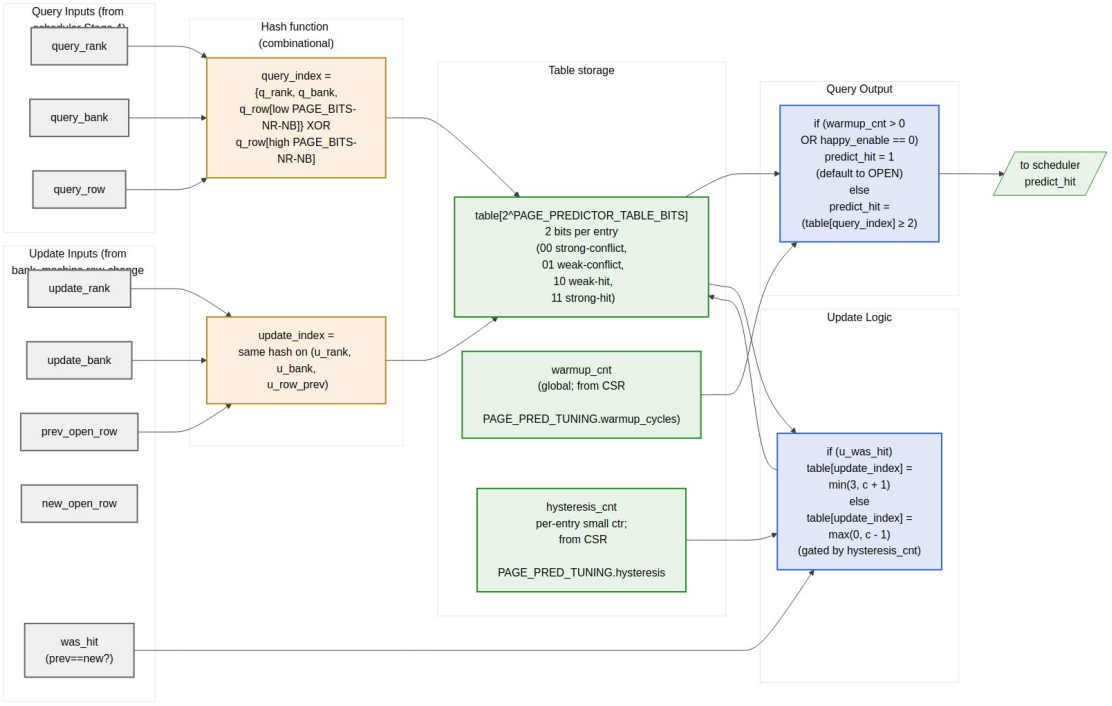
```
        assign predict_hit_to_sched = '0;        // always predict
"conflict" → RDA/WRA
    end
endgenerate
```

The scheduler's interface is identical in all three cases; the predictor's presence is invisible at the scheduler boundary.

13.3 Synthesis Parameters

Parameter	Source	Effect
PAGE_PREDICTOR_TABLE_BITS	top (default 12)	Table size = 2^PAGE_PREDICTOR_TABLE_BITS entries (default 4 K entries)
NUM_RANKS	top	Folded into the hash index
NUM_BANKS	top	Folded into the hash index
ROW_WIDTH	top	Source for the row-bit hash inputs
HYSTERESIS_BITS	derived	Width of per-entry hysteresis counter (small, e.g., 3 bits)
WARMUP_WIDTH	derived	Width of global warmup counter (16-bit per CSR field)

13.4 Block Implementation View



Page-Predictor Implementation — hash, table, warmup, hysteresis, query/update

Source: 09_page_predictor_implementation.mmd

The HAS-side algorithm view (`ddr2_lpddr2_has/assets/mermaid/09_happy_predictor.png`) shows the query/update *flow*; the diagram above shows the FUB's *physical organization* — input ports, hash, table, gate logic.

13.5 Hash Function

The table is indexed by a hash of (rank, bank, row). Multi-rank is folded into the same table — there is no per-rank replication. The hash is the **concatenation of identity bits with a XOR-folded row hash**:

```
hash_inputs    = {rank, bank, row}        // total width: clog2(NR) +
clog2(NB) + ROW_WIDTH
table_idx_width = PAGE_PREDICTOR_TABLE_BITS

// Identity portion: low (clog2(NR) + clog2(NB)) bits of the index
id_portion = {rank, bank}                  // width clog2(NR) +
clog2(NB)
```

```
// Hash portion: remaining (table_idx_width - id_portion_width) bits
of the index
hash_portion = row[low_HASH_BITS]
               XOR row[mid_HASH_BITS]
               XOR row[high_HASH_BITS]
               // (each slice is HASH_BITS = table_idx_width -
id_portion_width wide)

table_idx = {id_portion, hash_portion}
```

Concrete example at default config (`NR=1, NB=8, ROW_WIDTH=14, PAGE_PREDICTOR_TABLE_BITS=12`):

- `id_portion` = `{0-bit rank, 3-bit bank}` = 3 bits
- `hash_portion` = 9 bits = `row[8:0] XOR row[13:5]` (XOR of two row slices)
- `table_idx` = `{3-bit bank, 9-bit row_hash}` — 4 K entries, 8 banks each get a 512-entry slice of the table

For multi-rank (`NR=4`, others same):

- `id_portion` = `{2-bit rank, 3-bit bank}` = 5 bits
- `hash_portion` = 7 bits = `row[6:0] XOR row[13:7]`
- `table_idx` = `{2-bit rank, 3-bit bank, 7-bit row_hash}` — 4 K entries, 32 (rank, bank) tuples each get a 128-entry slice

Why fold rank into identity rather than hash. Folding rank into the identity portion guarantees that two different ranks never collide for the same row hash. The trade-off is that the per-(rank, bank) slice shrinks (128 entries each at NR=4 vs 512 at NR=1). At default 4 K-entry table, this is fine — even 128 entries per slice is more than enough to track the working set of a typical bank.

If `PAGE_PREDICTOR_TABLE_BITS` is small enough that the identity portion eats the whole index (e.g., `PAGE_PREDICTOR_TABLE_BITS = 5` at `NR=4, NB=8`), there is no `hash_portion` and the predictor degrades to a per-(rank, bank) single-counter. An elaboration assertion fires in that case requiring `PAGE_PREDICTOR_TABLE_BITS >= id_portion_width + 4`.

13.6 Table Storage

Each entry is a 2-bit saturating counter:

- `00` = strong conflict-likely
- `01` = weak conflict-likely
- `10` = weak hit-likely (reset value)
- `11` = strong hit-likely

A `predict_hit` query returns 1 when the entry value is `>= 2` (hit-likely side of the saturating range).

Total storage = `2^PAGE_PREDICTOR_TABLE_BITS × 2 bits`. Concrete:

PAGE_PREDICTOR_TABLE_BITS	Entries	Storage	Implementation
8	256	512 bit	Distributed flops
12 (default)	4 K	8 Kbit (1 KB)	Single BRAM18 (FPGA) / small RF (ASIC)
16	64 K	128 Kbit (16 KB)	Multiple BRAM tiles or SRAM macro

The storage choice is synthesis-script driven: a SystemVerilog `(* ram_style = "block" *)` attribute on the table array hints BRAM for sizes ≥ 8 (the BRAM18 sweet spot at 512×36-bit = 512 entries × 4 bits, which fits two predictor lookups per BRAM word).

At the default 12-bit (4 K entries), a single 36 Kbit BRAM18 holds the entire table with room to spare for hysteresis counters. The query path is one BRAM-read cycle latency (registered output) — important for the scheduler's Stage-4 timing.

BRAM port arrangement. The table has one read port (the query path) and one read-modify-write port (the update path). On 7-series BRAM, this is naturally a true dual-port configuration. Update and query can happen in parallel.

13.7 Warmup Counter

A global warmup counter delays the predictor's first trustworthy output. At reset:

```
warmup_cnt = PAGE_PRED_TUNING.warmup_cycles    // 16-bit, default 4096
```

While `warmup_cnt > 0`:

- Every cycle decrement `warmup_cnt`
- Query output forces `predict_hit_o = 1` (default open)

When `warmup_cnt == 0`:

- Query output is `(table[table_idx] >= 2)`

The warmup exists because the table starts at "weakly hit-likely" everywhere. Without it, the predictor would output uniform "hit" for the first few hundred accesses anyway — but the warmup makes that explicit and tunable. Software can shorten the warmup for short-running workloads or extend it for cold-start benchmarks.

13.8 Hysteresis Per Entry

To avoid the predictor flipping on every single misprediction, each table entry has a small **hysteresis counter** that gates updates:

```
struct entry_t {
    logic [1:0] saturating_counter;
    logic [HYSTERESIS_BITS-1:0] hyst_cnt;
};

// On update with was_hit:
//   if (hyst_cnt > 0):
//     hyst_cnt = hyst_cnt - 1
//     (no change to saturating_counter)
//   else:
//     update saturating_counter (inc on hit, dec on conflict)
//     hyst_cnt = PAGE_PRED_TUNING.hysteresis
```

At reset `hyst_cnt = 0` (full sensitivity). The CSR-supplied hysteresis value (default 0) is added per update. With `PAGE_PRED_TUNING.hysteresis = 3`, the predictor only updates every 4th conflicting observation — useful for workloads with high inherent variance.

The hysteresis flops are *per-entry* — at default 12-bit table with 3-bit hysteresis, total hysteresis storage is `4096 × 3 = 12 Kbit`, which is significant. The hysteresis_bits parameter defaults to 0 (disabling hysteresis); enabling it is opt-in via `HYSTERESIS_BITS` at elaboration. When `HYSTERESIS_BITS == 0` the FUB synthesizes no hysteresis flops and the update fires every cycle.

13.9 Update Path

The predictor is *trained* by `bank_machine_fub`'s open-row change broadcast (§2.9). Each time a bank's open row changes (via PRE+ACT or auto-precharge → ACT), the predictor learns:

```
was_hit = (new_open_row == prev_open_row_when_request_was_made)
        // i.e., did the next access actually hit the row we predicted
on?
```

This requires the predictor to remember the row context of each prediction it made. The simplest implementation: a small per-bank shadow register that captures the row each request was predicted on. On open-row change broadcast, the shadow register's row is the "prev" row that was predicted on, and the broadcast's new_open_row is what the actual row turned out to be.

The shadow register is `ROW_WIDTH × NR × NB` flops total — for default config (14 × 1 × 8) = 112 flops. For multi-rank (14 × 4 × 8) = 448 flops.

Why this works. The HAPPY paper trains on actual observed outcomes; we observe outcomes by watching bank state transitions. A row-hit observation occurs when the bank stays ACTIVE on the same row across multiple column commands; a row-conflict occurs when the bank transitions PRE → ACT (new row).

13.10 Query Path Timing

The scheduler's Stage-4 (per §2.7) queries the predictor when:

- An entry has been picked for issue
- Lookahead is inconclusive (no same-bank pending in the lookahead window)
- `PAGE_POLICY == HAPPY_HYBRID`

The query is single-cycle:

```
cycle T:   scheduler asserts query_valid_o, query_rank/bank/row
cycle T:   predictor computes hash, indexes table (BRAM read)
cycle T+1: predict_hit_o is valid
cycle T+1: scheduler latches the auto-precharge decision
```

This puts the predictor in the scheduler's Stage-4 register-to-register path. The BRAM read access dominates: at FPGA targets ~1.5 ns. Combined with the scheduler's Stage-4 mux, the total Stage-4 path is ~2.2 ns — fits the 200 MHz target (5 ns) with slack; the 500 MHz target requires an extra register stage on the predict_hit output.

If a query and an update target the same table entry in the same cycle, the BRAM's true-dual-port write-first behavior makes the read return the newly-written value. This is fine for correctness — the more-recent training applies.

13.11 Interface

13.11.1 Query (from scheduler)

Signal	Direction	Width	Description
`query_valid_i`	input	1	Scheduler is requesting a prediction
`query_rank_i`	input	`$clog2(NR)`	Query rank
`query_bank_i`	input	`$clog2(NB)`	Query bank
`query_row_i`	input	`ROW_WIDTH`	Row of the request being predicted

Signal	Direction	Width	Description
predict_hit_o	output	1	1-cycle-later: 1 = predicted hit, 0 = predicted conflict

13.11.2 Update (from bank machine broadcast)

Signal	Direction	Width	Description
bank_orw_changed_i	input	NR×NB	One-cycle strobe per (rank, bank) — from §2.9 broadcast
bank_new_open_row_i	input	NR×NB × ROW_WIDTH	New open row value

The predictor internally maintains the shadow `prev_open_row[NR][NB]` register described above; the strobe + new row let it compute `was_hit = (bank_new_open_row_i == prev_open_row_shadow[r][b])`.

13.11.3 CSR live inputs

Signal	Direction	Width	Source
cfg_warmup_i	input	16	PAGE_PRED_TUNING.warmup_cycles
cfg_hysteresis_i	input	8	PAGE_PRED_TUNING.hysteresis
cfg_happy_enable_i	input	1	SCHED_TUNING.happy_enable

13.11.4 Telemetry outputs

Signal	Direction	Width	Description
dbg_table_warm_pct_o	output	8	Fraction of table entries that have updated at least once
dbg_accuracy_per_bank_o[NR][NB]	output	8	Rolling prediction accuracy per (rank, bank)
dbg_warmup_active_o	output	1	warmup_cnt > 0

The accuracy telemetry is the **single most useful debug signal** in the FUB. The bring-up team will watch it to decide whether HAPPY is helping, hurting, or break-even on the workload of the day.

13.12 CSR Hooks

CSR field	Source	Use case
OBS_PAGE_PRED_ACCURACY (R)	Aggregate accuracy across all banks	Headline predictor health
OBS_PAGE_PRED_ACCURACY_R<R>_B<N> (R)	Per-(rank, bank) accuracy (sparse, mirrors OBS_BANK_OPEN_ROW_*)	Per-bank predictor health
STATUS.page_pred_warmup_active (R)	dbg_warmup_active_o	Is the predictor producing real predictions yet?
PAGE_PRED_TUNING.warmup_cycles (R/W)	cfg_warmup_i	Tunable warmup
PAGE_PRED_TUNING.hysteresis (R/W)	cfg_hysteresis_i	Tunable hysteresis

13.13 Verification Notes (cocotb test plan)

Scenario	What it proves
Cold reset; all queries return predict_hit = 1 (warmup phase)	Warmup gate works
warmup_cycles = 0; first query after reset goes through the table	Warmup can be disabled
warmup_cycles = 4096; query returns gate-default until cycle 4096	Warmup countdown
Train 100 row-hits to (rank 0, bank 3); predict_hit on a fresh query to same bank	Counter trained to strong-hit
Train 100 row-conflicts to (rank 0, bank 3); predict_hit returns 0	Counter trained to strong-conflict
hysteresis = 3; counter updates only every 4th observation	Hysteresis gate
Multi-rank (NR=2): training on (0, b) does not affect (1, b)	Rank-isolation via identity bits
Two different rows on same bank with identical low-bits collision	Hash sensitivity to row mid/high bits

Scenario	What it proves
`happy_enable = 0` runtime; queries return 1 regardless of table state	Runtime kill-switch
`PAGE_POLICY != HAPPY_HYBRID` build: FUB not instantiated	Conditional gen check
Concurrent query + update to same hash bucket	BRAM true-dual-port write-first read returns new value
Predictor accuracy CSR telemetry tracks ground truth	Accuracy counter correctness

13.14 Open Questions / Future Work

- **Per-bank vs unified table.** A unified table (current design) shares storage across all (rank, bank) pairs. An alternative is a per-bank smaller table — saves cross-bank interference at the cost of less aggregate storage. The unified design's identity-bits scheme already provides per-(rank, bank) isolation at the index level; per-bank tables would only help if the row hash collisions are concentrated. Punt; revisit if characterization shows clustering.
- **Update gating during refresh.** When a bank's open-row changes due to refresh (REFRESHING → IDLE → ACT), the predictor sees a row-change broadcast that isn't really a "prediction outcome." Currently the update path doesn't distinguish. A flag bit on the broadcast (`is_refresh_induced`) would let the predictor skip these updates. Cheap to add; not in v1.
- **Two-level predictor.** HAPPY in the paper is a two-level predictor (history table + decision table). The v1 implementation is the one-level decision-table only — simpler and the paper's results show one-level captures most of the benefit. The two-level variant could be added as `PAGE_PREDICTOR_LEVELS = 2` parameter if characterization shows v1 misses the workload's true pattern.
- **Predictor accuracy across runtime mode changes.** When `force_inorder` toggles or `lookahead_active` changes, the predictor's training set distribution shifts. Should the accuracy counters reset on these mode changes? Probably yes — adds one extra clear strobe. Punt to v2 unless bring-up calls it out.

14 Bank Machine (bank_machine_fub)

Module: bank_machine_fub.sv **Location:** rtl/fub/ **Category:** FUB **Parent:** ddr2_lpddr2_ctrl (instantiated NUM_RANKS × NUM_BANKS times) **Status:** Draft v0.1

> Architectural context: HAS §3.3. The FSM state diagram is in ddr2_lpddr2_has/assets/mermaid/03_bank_machine_fsm.png and is the canonical state reference — this section is the implementation view (port interface, counter pool, broadcast updates, multi-rank instantiation, timing).

14.1 Purpose

bank_machine_fub is the per-(rank, bank) state machine that enforces JEDEC per-bank timing constraints. One FSM instance per (rank, bank) — so a 1-rank 8-bank build has 8 instances, a 2-rank 8-bank build has 16, a 4-rank 8-bank build has 32. Each instance tracks:

- Open-row state and the row identifier
- All per-bank timing counters (tRCD, tRAS, tRP, tRC, tWR, tRFCpb, plus per-RD CL and per-WR CWL windows)
- The “last refreshed” age used by DARP per-bank refresh selection
- The refresh handshake state to refresh_mgr_fub

The instance fans its outputs into:

- A per-(rank, bank) row of the scheduler’s bank-state matrix
- The txn_queue’s row-hit-cache refresh broadcast (open_row_changed[r][b] + new_open_row[r][b])
- The refresh manager’s refresh_gnt[r][b] array and the last_ref_age[r][b] DARP selection input

The bank machine is the most-replicated FUB in the design. Per-instance area is small (~150–300 LUTs, see below), but the replication factor (NR × NB, up to 32) makes total bank-machine area the second-largest after the scheduler.

14.2 Synthesis Parameters

Parameter	Source	Effect on this FUB
ROW_WIDTH	top	Width of open_row register and new_open_row broadcast
COL_WIDTH	top	Width of column passthrough (consumed by cmd_encoder, not stored)
MEMTYPE	top	Controls whether the REFRESHING state uses tRFCab or tRFCpb (LPDDR2 only has REFpb)
T_*_WIDTH_MAX	derived	Width of each timing-counter register (set by max CSR-loaded tREFI, tRC, tRFCpb, etc.)
LAST_REF_AGE_WIDTH	derived	Width of last_ref_age counter; sized to span 8 × tREFI_cycles without saturation

The bank machine takes **no rank/bank identifier as a parameter** — every instance is identical. Identity comes from the position in the top-level generate block and is passed in via wire-level bank_id_i and rank_id_i ports (used only for assertion messages and CSR routing; the FSM logic is identity-agnostic).

14.3 Instantiation Pattern

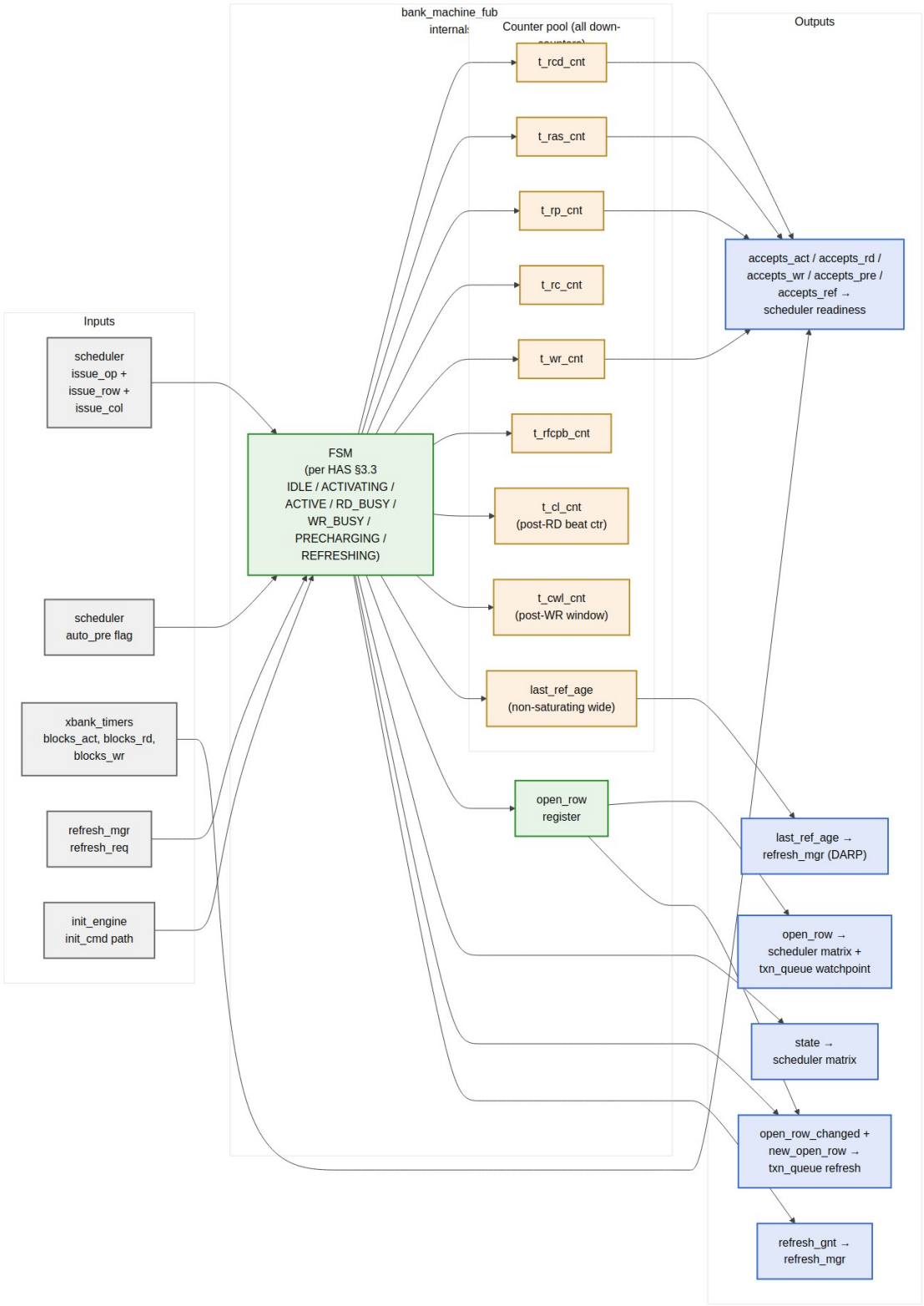
Recap from §2.1 top-integration:

```
generate
    for (genvar r = 0; r < NUM_RANKS; r++) begin : g_rank
        for (genvar b = 0; b < NUM_BANKS; b++) begin : g_bank
            bank_machine_fub #(.ROW_WIDTH(ROW_WIDTH)) u_bank_machine (
                .mc_clk             (mc_clk),
                .mc_rst_n           (mc_rst_n),
                .rank_id_i          (r[$clog2(NR)-1:0]),
                .bank_id_i          (b[$clog2(NB)-1:0]),
                .sched_issue_if     (sched_to_bank[r][b]),
                .xbank_blocks_i     (xbank_to_bank[r][b]),
                .refresh_req_i      (refresh_req[r][b]),
                .refresh_gnt_o      (refresh_gnt[r][b]),
                .init_cmd_if        (init_to_bank[r][b]),
                .state_o            (bank_state[r][b]),
                .open_row_o         (bank_open_row[r][b]),
                .accepts_o          (bank_accepts[r][b]),
                .open_row_changed_o (orw_changed[r][b]),
                .new_open_row_o     (new_orw[r][b]),
                .last_ref_age_o     (bank_last_ref_age[r][b])
```

```
            );
        end
    end
endgenerate
```

The outputs are 2-D-indexed arrays at the top level. The scheduler and txn_queue consume them by name; no aggregation logic in the top — pure structural wiring.

14.4 Block Internals



Bank-Machine Internals — port boundary, counter pool, FSM, broadcasts

Source: 07_bank_machine_internals.mmd

The diagram above is the implementation view. The HAS state-diagram view is at `ddr2_lpddr2_has/assets/mermaid/03_bank_machine_fsm.png` — refer to it for state transitions; this section is the port + counter detail.

14.5 FSM States and Transition Triggers

Recap of the seven states (full state diagram in HAS):

State	Held while	Exits via
`IDLE`	No row open; bank free	ACT or REFpb issued by scheduler
`ACTIVATING`	`t_rcd_cnt > 0`	`t_rcd_cnt == 0` → `ACTIVE`
`ACTIVE`	Row open; awaiting column command or PRE	RD/RDA, WR/WRA, or PRE issued by scheduler
`RD_BUSY`	`t_cl_cnt > 0` (CL + BL/2 beats remaining)	Last beat returned → `ACTIVE` (bare RD) or `PRECHARGING` (RDA)
`WR_BUSY`	`t_cwl_cnt > 0` (CWL + BL/2 + tWR window)	Window expired → `ACTIVE` (bare WR) or `PRECHARGING` (WRA)
`PRECHARGING`	`t_rp_cnt > 0`	`t_rp_cnt == 0` → `IDLE`
`REFRESHING`	`t_rfc_cnt > 0` (tRFCpb for LPDDR2, tRFCab for DDR2 REFab)	`t_rfc_cnt == 0` → `IDLE`

Auto-precharge handling. RDA / WRA do not require a separate state — they are encoded as an "auto-pre pending" flag set when the FSM enters `RD_BUSY` / `WR_BUSY`. When the busy window expires, the flag steers the next-state to `PRECHARGING` instead of `ACTIVE`. No PRE command is consumed from the scheduler — the precharge timing is purely internal. This is the same trick the cmd_encoder uses to fold RDA into one DRAM command.

Init-engine bypass. During init, the `init_cmd_if` path can issue ACT, PRE, MRS, REF directly without going through the scheduler. The FSM honors these as if they came from the scheduler — same state transitions, same counter reloads. The init engine knows the JEDEC sequence; the bank machine just executes it.

14.6 Counter Pool

All counters are **down-counters that saturate at 0**. Each is loaded with a CSR-supplied cycle count on the relevant trigger.

Counter	Load Trigger	Reload Value (cycles)	Saturates At	Width
t_rcd_cnt	ACT issued	TIMINGS_RC_RCD_RP.tRCD	0	8 bits
t_ras_cnt	ACT issued	TIMINGS_RC_RCD_RP.tRAS	0	8 bits
t_rp_cnt	PRE or PREA issued; or RDA/WRA window expiration	TIMINGS_RC_RCD_RP.tRP	0	8 bits
t_rc_cnt	ACT issued	TIMINGS_RC_RCD_RP.tRC	0	8 bits
t_wr_cnt	WR or WRA last beat issued	TIMINGS_CL_CWL_WR.tWR	0	8 bits
t_rfcpb_cnt	REF or REFpb issued	LPDDR2 tRFCpb / DDR2 tRFCab	0	16 bits
t_cl_cnt	RD or RDA issued	TIMINGS_CL_CWL_WR.CL + BL/2	0	8 bits
t_cwl_cnt	WR or WRA issued	TIMINGS_CL_CWL_WR.CWL + BL/2 + tWR	0	8 bits
last_ref_age	REF or REFpb completion (t_rfcpb_cnt → 0)	reload to 0; otherwise +1 per cycle	does not saturate (wide enough for 8 × tREFI)	24 bits

The CSR-supplied values are read from TIMINGS_* registers in csr_apb_fub; the values are live (not staged) since timing parameters are *typically* loaded once at init and not changed at runtime. Software *can* change them at quiet points, but the bank machine doesn't enforce quiet-point staging — it just samples them on each counter load.

`last_ref_age` is non-saturating because DARP needs to distinguish an entry that was refreshed 10 cycles ago from one refreshed 10,000 cycles ago. A 24-bit width supports `2^24 / 200 MHz = 84 ms` of age, well past 8 × tREFI (typically 50 µs).

14.7 Outputs to Scheduler (`accepts_*`)

The `accepts_o` struct is the **combinational readiness output** consumed by the scheduler's Stage-1 readiness computation. Each member is the AND of "FSM state permits this command" and "all relevant counters are 0" and "xbank constraints permit this command":

```
accepts_act = (state == IDLE)
            AND (t_rp_cnt == 0)
            AND (t_rc_cnt == 0)
            AND NOT xbank_blocks_act

accepts_rd  = (state == ACTIVE)
            AND (t_rcd_cnt == 0)
            AND NOT xbank_blocks_rd

accepts_wr  = (state == ACTIVE)
            AND (t_rcd_cnt == 0)
            AND NOT xbank_blocks_wr

accepts_pre = (state == ACTIVE)
            AND (t_ras_cnt == 0)

accepts_ref = (state == IDLE)
            AND (t_rp_cnt == 0)
            AND (t_rc_cnt == 0)
```

These are pure combinational — no flop between the state register and the scheduler. The scheduler's Stage-1 critical path (per §2.7) routes through these comparators.

Why xbank lives outside. Cross-bank constraints (tRRD, tFAW, tCCD, tWTR, tRTW) are shared across all banks of a rank or globally across the channel — they can't be enforced inside a per-bank FSM. The `xbank_timers_fub` (§2.10) aggregates them and presents the per-(rank, bank) AND-mask via `xbank_blocks_i`.

14.8 Refresh Handshake Protocol

The bank machine cooperates with `refresh_mgr_fub` via a two-wire handshake per (rank, bank):

Signal	Direction	Meaning
refresh_req_i	refresh_mgr → bank	"Drain to a quiet state and await my REF / REFpb command"
refresh_gnt_o	bank → refresh_mgr	"I am in IDLE (or REFRESHING) and will not issue any column command"

The bank machine's protocol:

1. **refresh_req asserts** while the bank is in ACTIVE / RD_BUSY / WR_BUSY: the bank does not assert refresh_gnt yet. It finishes its current column command (if any) and falls back to ACTIVE. The scheduler is masked from issuing new column commands to this bank during the refresh window (txn_queue watchpoint — see §2.6 and §2.7).
2. **refresh_req asserts** while the bank is ACTIVE and no column command is in flight: the bank issues an internal precharge (driving accepts_pre high to signal the scheduler can issue PRE; or, if the scheduler is gated for the refresh window, the bank can self-issue PRE on t_ras_cnt == 0). Once state == IDLE and t_rp_cnt == 0, refresh_gnt asserts.
3. **refresh_req asserts** while the bank is in IDLE: refresh_gnt asserts the same cycle.
4. **REF/REFpb arrives** from the scheduler (via the refresh manager's priority path): FSM transitions IDLE → REFRESHING, t_rfcpb_cnt loads, last_ref_age resets to 0.
5. **refresh_req deasserts** after refresh completes: bank machine drops refresh_gnt, transitions REFRESHING → IDLE on t_rfcpb_cnt == 0, and is free to accept new ACTs.

For multi-rank REFab dispatch (per §2.11 refresh_mgr), the refresh manager asserts refresh_req[r][*] for all banks on the target rank simultaneously and waits for all refresh_gnt[r][*] before issuing REF. Other ranks' bank machines see refresh_req[r' ≠ r] = 0 throughout and continue normal operation.

14.9 Open-Row Change Broadcast

When the FSM transitions in a way that changes the open row, the bank machine asserts a one-cycle broadcast to txn_queue_fub:

Transition	open_row_changed_o	new_open_row_o
IDLE → ACTIVATING (via ACT)	1 cycle	The ACT's row
ACTIVE → PRECHARGING → IDLE (after RDA/WRA or explicit PRE)	1 cycle (on entering IDLE)	0 (no row open)
REFRESHING → IDLE	1 cycle	0 (no row open)
ACTIVATING → ACTIVE	not asserted (row was already valid as of ACTIVATING)	—

This is the path that drives `txn_queue`'s `row_hit_cached` refresh, as described in §2.6. The one-cycle lag (broadcast at T, queue refresh visible at T+1) is the deliberate trade described in both this block and the scheduler (§2.7).

`new_open_row_o = 0` when the bank is leaving an open-row state is a convention — the queue's refresh path compares against the new row only when the bank ends in an active state. When ending in IDLE, the queue sets `row_hit_cached = 0` for all entries targeting this (rank, bank) regardless of `new_open_row_o`.

14.10 Per-Instance vs. Aggregated Storage

What lives per-instance (NR × NB copies):

- `state` register (3 bits)
- `open_row` register (ROW_WIDTH bits)
- All counters (~80 bits per instance)
- `auto_pre_pending` flag (1 bit)

What is aggregated at the top level (one copy each):

- `bank_state[NR][NB]` — concatenation of per-instance `state` outputs into a 2-D array
- `bank_accepts[NR][NB]` — concatenation of `accepts` structs
- The DARP candidate-set vector (computed by `refresh_mgr_fub` from the array of `last_ref_age` outputs)
- The scheduler's match vector (computed by the scheduler from `bank_state` and `bank_accepts`)

This separation matters for synthesis: the per-instance flops are local to each bank machine (good for placement); the aggregation buses are wide (NR × NB × ~30 bits = up to ~1000 bits at 4×8) and route across the chip to the scheduler and refresh manager.

Routing concern. At `NUM_RANKS=4`, `NUM_BANKS=8`, the bank-state array routes 32 instances' worth of state and accepts wires to the scheduler. This is a ~1000-bit bus and is the second-widest in the design (after `txn_queue.q_entries_o`). The scheduler's Stage-1 critical path absorbs this in the 200 MHz target with slack; the 500 MHz target requires registered bank-state outputs (which adds one cycle to the scheduler-to-issue latency but is otherwise harmless — bank states change slowly compared to scheduler issue).

14.11 Per-Instance Area Estimate

At default config (ROW_WIDTH=14, COL_WIDTH=10, 8-bit timing counters, 24-bit `last_ref_age`):

Item	Per instance
FSM state (3 bits + 3-bit next)	~10 LUTs
Counter pool (8 down-counters)	~60 flops + 30 LUTs
`open_row` register (14 bits)	14 flops
`last_ref_age` (24 bits)	24 flops
`auto_pre_pending` + combinational `accepts_*` glue	~20 LUTs
Per-instance total	**~120 flops + 60 LUTs ≈ 100–150 LUTs**
32-instance total (4-rank × 8-bank)	**~3,200–4,800 LUTs**

This is ~25% of the controller's total LUT budget at the max-rank config — significant. At single-rank, the bank machines fall to ~800–1,200 LUTs, comfortably in budget.

14.12 CSR Hooks

CSR field	Source signal	Use case
`OBS_ROW_HIT_RANK<R>_BANK<N>` (R)	Driven by scheduler when row-hit picks this bank	Per-bank row-hit telemetry (HAS §6.3)
`OBS_REF_LATENCY_RANK<R>_BANK<N>` (R)	Time from `refresh_req` to `refresh_gnt`	Per-bank refresh-blocking time
`OBS_BANK_OPEN_ROW_R<R>_B<N>` (R)	Current `open_row` value	Debug: which row is currently open

CSR field	Source signal	Use case
OBS_BANK_STATE_R<R>_B<N> (R)	Current FSM state (3-bit encoding)	Debug: bank state for bring-up
STATUS.bank_idle_mask (R)	One bit per (rank, bank): state == IDLE	Quick visibility into bank availability

The per-(rank, bank) observation registers are sparse at multi-rank — at NR=4, NB=8 there are 32 of each. The CSR slave generates them from a YAML template (see §4.2). Software queries the cap_max_ranks and NUM_BANKS capability bits (§4.4) to know which registers exist.

14.13 Verification Notes (cocotb test plan)

Scenario	What it proves
Single ACT → RD → PRE → IDLE on bank 0	Smoke: each counter loads and decrements correctly
ACT → RDA: bank auto-precharges without explicit PRE from scheduler	Auto-precharge flag steers RD_BUSY → PRECHARGING
ACT → WRA: bank auto-precharges; t_wr_cnt window respected	Write recovery before precharge
Scheduler issues RD while t_rcd_cnt > 0	Assertion fires; accepts_rd was 0 — should be impossible
refresh_req asserts during ACTIVATING	refresh_gnt waits until bank reaches IDLE
refresh_req asserts during RD_BUSY	Bank completes RD, returns to ACTIVE, then drains via PRE
refresh_req asserts when bank is IDLE	refresh_gnt asserts same cycle
REF completion → last_ref_age resets to 0	DARP age-tracking correctness
Two adjacent ACTs to different rows on same bank — second blocks on t_rp_cnt	Per-bank tRP enforcement
Cross-bank ACT (handled by xbank_timers) — should NOT affect this bank	Per-bank FSM is rank/bank-local
Multi-rank (NR=2): refresh on rank 0 bank 3; rank 1 bank 3 continues normal ops	Per-rank refresh isolation
Open-row change broadcast: ACT issued, open_row_changed asserts for 1 cycle	Broadcast timing correctness

Scenario	What it proves
Soft reset during `WR_BUSY` — counters clear, state returns to IDLE	Reset behavior

14.14 Open Questions / Future Work

- **CSR coverage of per-bank state.** At NR=4, NB=8 the `OBS_BANK_OPEN_ROW_*` registers consume 32 CSR slots — non-trivial. Worth gating behind a `cap_per_bank_obs` capability bit so smaller builds skip them entirely?
- **Per-instance vs centralized `last_ref_age`.** Could the age counters live in `refresh_mgr_fub` and free up flops per bank machine? Slight area win but adds a wide read port from refresh_mgr to bank_machine for age comparison; punt to v2.
- **Auto-pre flag clear timing.** Currently the `auto_pre_pending` flag clears on entering `PRECHARGING`. If a refresh request arrives mid-RD with auto-pre pending, the bank still does the auto-pre (drains via tRP) before granting refresh. This is correct per JEDEC, but worth a verification scenario (added above).
- **tWR vs tRP composition.** A WRA → PRECHARGING transition needs `max(t_wr_cnt, t_rp_load)` cycles. Implementation: the FSM enters PRECHARGING after `t_wr_cnt` expires and *then* loads `t_rp_cnt`. This is the simplest correct sequence; alternative is to start `t_rp_cnt` early and gate `IDLE` on `max(t_wr_cnt, t_rp_cnt)`. The simpler form is preferred unless characterization shows it costs measurable bandwidth.

---

15 Cross-Bank Timers (`xbank_timers_fub`)

Module: `xbank_timers_fub.sv` **Location:** `rtl/fub/` **Category:** FUB **Parent:** `ddr2_lpddr2_ctrl` (one instance) **Status:** Draft v0.1

Architectural context: HAS §3.3 `xbank_timers` and the “Per-Rank vs. Cross-Rank Scope” table added in v0.2. This block-level MAS section is

the implementation view: per-constraint tracker structure, the per-rank vs global scope partition, multi-rank cross-rank gates, and the combinational `blocks_*` outputs.

15.1 Purpose

`xbank_timers_fub` enforces the **cross-bank** JEDEC timing constraints — the ones that span banks of the same rank, or span ranks of the same channel. Per-bank constraints (tRCD, tRAS, tRP, etc.) live in `bank_machine_fub` (§2.9); this FUB owns everything that cannot live in a single bank machine because the relevant resource (the DRAM device's ACT-rate budget, the shared DQ bus, the chip-select setup) is shared.

The FUB produces per-(rank, bank) AND-mask signals (`blocks_act`, `blocks_rd`, `blocks_wr`) that gate each bank machine's `accepts_*` outputs. These masks are the bridge between cross-bank reality and the per-bank state machines' otherwise-local view.

There is **one instance** at the top level — even at multi-rank, the FUB is internally partitioned by scope rather than replicated.

15.2 Constraint Inventory

Per HAS §3.3, the cross-bank constraints partition into three scopes:

Constraint	Scope	Why
`tRRD`	per-rank	Limits ACT-to-ACT stagger inside one DRAM device
`tFAW`	per-rank	Four-Activate-Window — device-local thermal/power limit
`tCCD`	global (channel)	Column-to-column on shared DQ bus
`tWTR`	global (channel)	Write-data → read-data turnaround on shared DQ bus
`tRTW`	global (channel)	Read-data → write-data turnaround on shared DQ bus
`tRTRS`	cross-rank only (NR > 1)	Rank-to-rank DQ-bus handoff on consecutive reads to different ranks
`tCS`	cross-rank only (NR > 1)	Chip-select setup when issuing a command to a different rank than the last

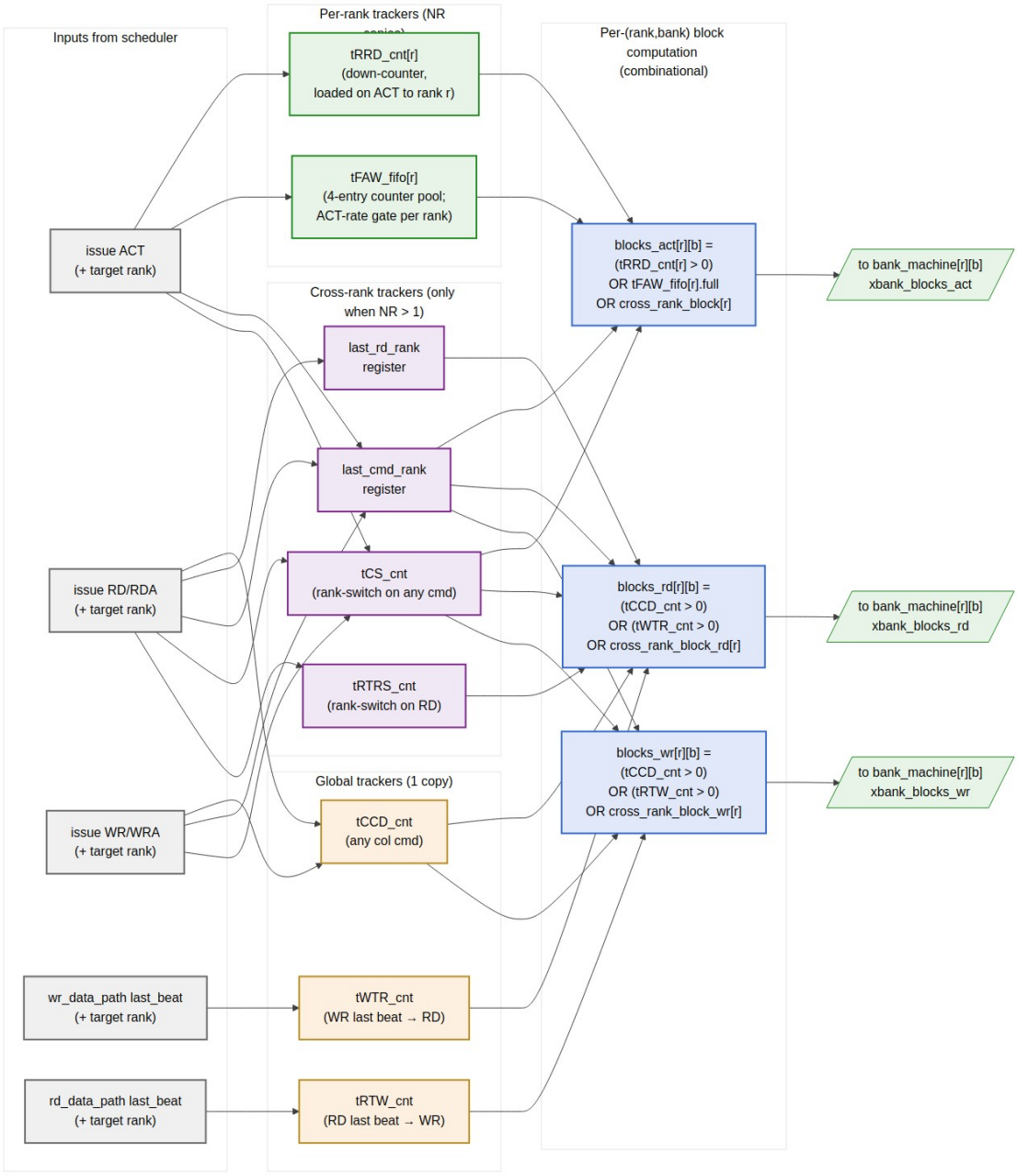
For NUM_RANKS == 1 builds, the cross-rank section is fully omitted at elaboration — the tRTRS_cnt and tCS_cnt flops are not synthesized and the last_rd_rank / last_cmd_rank registers tie off. The other tracker categories are present in every build.

15.3 Synthesis Parameters

Parameter	Source	Effect
NUM_RANKS	top	Per-rank tracker fanout; enables cross-rank gates when > 1
NUM_BANKS	top	Output fanout: blocks_*[NR][NB]
T_RRD_WIDTH	derived	Width of tRRD_cnt (sized to span max CSR-loaded tRRD value)
T_FAW_WIDTH	derived	Width of each tFAW_fifo counter
T_CCD_WIDTH	derived	Width of global tCCD_cnt
T_WTR_WIDTH	derived	Width of global tWTR_cnt
T_RTW_WIDTH	derived	Width of global tRTW_cnt
T_RTRS_WIDTH	derived (NR>1)	Width of tRTRS_cnt (typically 2-bit for DDR2/3 1-cycle tRTRS)
T_CS_WIDTH	derived (NR>1)	Width of tCS_cnt (typically 1-cycle on Artix-class FPGA targets)

The T_*_WIDTH parameters are computed at elaboration from the maximum CSR-loadable timing value. CSR values themselves are live (loaded once at init) and feed the counter-load values directly.

15.4 Block Internals



xbank_timers Constraint Tracker Pool

Source: 08_xbank_timers_scope.mmd

The diagram shows three tracker pools (per-rank, global, cross-rank), the scheduler-issue inputs that load them, and the per-(rank, bank) `blocks_*` combinational outputs.

15.5 Per-Rank Trackers

15.5.1 tRRD_cnt[NR]

One down-counter per rank.

- **Load**: `issue_op_o == ACT` on rank `r` → `tRRD_cnt[r] = TIMINGS_RRD_FAW_WTR.tRRD`
- **Decrement**: `tRRD_cnt[r] > 0` decrements every cycle
- **Effect**: contributes to `blocks_act[r][*]` (gates *all banks* of rank `r`, since tRRD spans banks within the rank)
- **No cross-rank interference**: an ACT on rank 0 does not affect `tRRD_cnt[1]`

At the default tRRD ~5 cycles (DDR2-800), the counter is 3 bits. At 200 MHz the load-to-zero latency is 5 cycles — well within the scheduler's per-cycle reckoning.

15.5.2 tFAW_fifo[NR]

Four-Activate-Window per rank. Implementation is a 4-entry counter pool (not a true FIFO of timestamps):

```
struct tFAW_slot_t {
    logic [T_FAW_WIDTH-1:0] cnt;   // down-counter; 0 means "this slot
is free"
};

tFAW_slot_t tFAW_fifo[NUM_RANKS][4];

// Per cycle, per slot:
//   if (cnt > 0) cnt = cnt - 1;

// On ACT issue to rank r:
//   pick the slot in tFAW_fifo[r] with the smallest cnt (the next-to-
evict slot;
//   one of them is guaranteed to be 0 if blocks_act_faw is currently
low)
//   load that slot's cnt = TIMINGS_RRD_FAW_WTR.tFAW

// blocks_act_faw[r] = AND over all 4 slots: (cnt > 0)
```

The "pick smallest" load policy is equivalent to "replace the oldest entry," which is correct for a rolling-window tFAW check. The check `AND over 4 slots cnt > 0` is "all 4 most-recent ACTs are still within tFAW" — block.

Why not a literal FIFO of timestamps. A timestamp FIFO requires subtraction at check time (`now - oldest_ts > tFAW`?) and a wide subtractor. The counter-pool approach is a 4-entry comparison-to-zero, which is one LUT layer.

At default tFAW ~32 cycles (DDR2-800), each slot's counter is 5 bits. Total `tFAW_fifo` storage per rank: 4 × 5 = 20 flops. Across 4 ranks: 80 flops. Trivial.

15.6 Global Trackers (channel-wide)

15.6.1 `tCCD_cnt`

Single down-counter. Tracks the minimum gap between *any* column command (RD, RDA, WR, WRA) and the *next* column command anywhere on the channel.

- **Load**: any column command issue → `tCCD_cnt = TIMINGS_RRD_FAW_WTR.tCCD`
- **Effect**: contributes to both `blocks_rd[*][*]` and `blocks_wr[*][*]`

At DDR2-800 with BL=4, tCCD = 2 cycles. The counter is 2 bits.

15.6.2 `tWTR_cnt`

Single down-counter. Tracks the minimum gap between the **last write data beat** completing and the **next read** issuing.

- **Load**: `wr_data_path_fub` strobes `wr_last_beat` → `tWTR_cnt = TIMINGS_RRD_FAW_WTR.tWTR`
- **Effect**: contributes to `blocks_rd[*][*]`
- **Subtle point**: loaded from the data-path event, *not* from the WR command issue. This is because tWTR is "last write data on DQ → first read CAS," not "WR command → RD command." The two are different by CWL + BL/2 cycles.

15.6.3 `tRTW_cnt`

Symmetric to tWTR. Tracks the minimum gap between the **last read data beat** completing and the **next write** issuing.

- **Load**: `rd_data_path_fub` strobes `rd_last_beat` → `tRTW_cnt = TIMINGS_CL_CWL_WR.tRTW` (CSR field added in v0.2)
- **Effect**: contributes to `blocks_wr[*][*]`

At DDR2-800 tRTW is typically 0 cycles, but the counter is still synthesized for parameterization and verification consistency. At DDR3+ tRTW becomes a relevant constraint; we want the FUB to be ready.

15.7 Cross-Rank Trackers (only when NUM_RANKS > 1)

These trackers synthesize only when `NUM_RANKS > 1` — at single-rank, they tie off and consume no flops.

15.7.1 `last_rd_rank` register

`$clog2(NR)`-bit register holding the rank of the most recent RD/RDA issue. Used to detect rank-switching for the tRTRS gate.

15.7.2 `tRTRS_cnt`

Single down-counter. Tracks the rank-to-rank read switching delay. DDR2 spec: 1 cycle when consecutive RDs target different ranks; 0 when same rank.

- **Load**: RD/RDA issued to rank `r`, and `r != last_rd_rank` → `tRTRS_cnt = TIMINGS_RTRS.tRTRS`
- **Effect**: contributes to `blocks_rd[r'][*]` *for* *`r' != last_rd_rank`* (block reads to a different rank while the bus handoff window is open)
- **Reset rule**: when an intervening WR or PRE breaks the read chain, `tRTRS_cnt` does not need to fire on the next RD. Implemented by also clearing on any non-RD issue.

15.7.3 `last_cmd_rank` register and `tCS_cnt`

Similar pair for the chip-select setup constraint. When any new command targets a rank different from the previous command, tCS-cycles must pass before the new `CS_n[r']` can drive.

- **Load**: any command issue where target rank `r != last_cmd_rank` → `tCS_cnt = TIMINGS_CS.tCS`
- **Effect**: contributes to `blocks_act[r'][*]`, `blocks_rd[r'][*]`, `blocks_wr[r'][*]` *for* *`r' != last_cmd_rank`*

On Artix-class FPGAs targeting DDR2-800, `tCS` is typically met by IOB setup margin and the counter loads with 0 cycles. The flop and gate exist for parameter-driven verification and for stricter-margin platforms.

15.8 Combinational blocks_* Outputs

The final per-(rank, bank) outputs are pure ORs of the relevant tracker bits:

```
// Common cross-rank block gates (only when NR > 1; else 0)
cross_rank_block[r] = (NR > 1)
                      AND (tCS_cnt > 0)
                      AND (r != last_cmd_rank)

cross_rank_block_rd[r] = cross_rank_block[r]
                       OR ( (NR > 1)
                            AND (tRTRS_cnt > 0)
                            AND (r != last_rd_rank) )

cross_rank_block_wr[r] = cross_rank_block[r]    // no tRTW-rank-switch
gate in v1

// Per-(rank, bank) outputs (same for all banks of a rank)
blocks_act[r][b] = (tRRD_cnt[r] > 0)
                OR (tFAW_fifo[r].all_busy)
                OR cross_rank_block[r]

blocks_rd[r][b] = (tCCD_cnt > 0)
               OR (tWTR_cnt > 0)
               OR cross_rank_block_rd[r]

blocks_wr[r][b] = (tCCD_cnt > 0)
               OR (tRTW_cnt > 0)
               OR cross_rank_block_wr[r]
```

Note that `blocks_*[r][b]` does not depend on `b` — every bank within a rank sees the same xbank gate. The 2-D `blocks_*` output is a fanout convenience for the top-level wiring; the underlying logic is `NR`-wide, then broadcast to `NB` banks.

This is a deliberate fanout: the bank machine port (§2.9) accepts `xbank_blocks_i` as a per-(rank, bank) struct because the wider port is symmetric with the bank machine's other 2-D-indexed signals. The cost is one extra wire-fanout layer at the top; the benefit is the bank-machine module declaration doesn't have to be parameterized differently.

15.9 Interface

15.9.1 Inputs from Scheduler

Signal	Direction	Width	Description
issue_op_i	input	4	Same encoding as scheduler_fub.issue_op_o
issue_rank_i	input	$clog2(NR)	Rank being issued to
issue_strobe_i	input	1	(issue_op_i != NOP) && issue_valid

15.9.2 Inputs from Data Paths

Signal	Direction	Width	Description
wr_last_beat_i	input	1	Last beat of a WR burst pushed to DFI wrdata
wr_last_rank_i	input	$clog2(NR)	Rank of that burst
rd_last_beat_i	input	1	Last beat of an RD burst returned from DFI rddata
rd_last_rank_i	input	$clog2(NR)	Rank of that burst

15.9.3 Inputs from CSR (live)

Signal	Direction	Width	Source
t_rrd_i	input	width	TIMINGS_RRD_FAW_WTR.tRRD
t_faw_i	input	width	TIMINGS_RRD_FAW_WTR.tFAW
t_ccd_i	input	width	TIMINGS_RRD_FAW_WTR.tCCD
t_wtr_i	input	width	TIMINGS_RRD_FAW_WTR.tWTR
t_rtw_i	input	width	TIMINGS_CL_CWL_WR.tRTW
t_rtrs_i	input	4	TIMINGS_XRANK.tRTRS (only when NR>1)
t_cs_i	input	4	TIMINGS_XRANK.tCS (only when NR>1)

15.9.4 Outputs to Bank Machines

Signal	Direction	Width	Description
`blocks_act_o[NR][NB]`	output	NR×NB	Per-(rank, bank) ACT gate
`blocks_rd_o[NR][NB]`	output	NR×NB	Per-(rank, bank) RD gate
`blocks_wr_o[NR][NB]`	output	NR×NB	Per-(rank, bank) WR gate

15.9.5 Debug Outputs

Signal	Description
`dbg_t_rrd_obs_o[NR]`	Live `tRRD_cnt[r]` values for waveform-debug
`dbg_tfaw_busy_count_o[NR]`	Per-rank count of non-zero tFAW slots
`dbg_xrank_block_active_o`	One-bit: any cross-rank gate currently asserted

15.10 Timing Budget

The path from scheduler issue → `blocks_*` update is single-cycle:

```
cycle T:   scheduler asserts issue_strobe_i with issue_op_i = ACT,
issue_rank_i = r
cycle T:   load tRRD_cnt[r], push tFAW_fifo[r], load tCS_cnt (if rank-
switched)
cycle T+1: tRRD_cnt[r] now > 0, blocks_act_o[r][*] = 1
```

The `blocks_*_o` outputs are combinational from the registers; the scheduler's Stage-1 path consumes them in the *same* cycle they update. So the scheduler always sees the post-issue gate state on the next cycle — no race where the scheduler picks two same-rank ACTs back-to-back.

For 500 MHz targets, the per-(rank, bank) fan-out path (NR × NB destinations) may need registered outputs. The 200 MHz default budget is comfortable.

Per-bank fan-out concern. At NR=4, NB=8, each `blocks_*` signal fans out 32 ways. Synthesis will buffer this automatically; no manual replication needed in RTL.

15.11 CSR Hooks

CSR field	Source	Use case
`STATUS.xbank_blocki`	Rolling count of cycles	Bandwidth-headroom

CSR field	Source	Use case
ng_pct (R)	where ANY blocks_* is asserted	telemetry
OBS_TFAW_HITS_RANK<R> (R)	Incremented when ACT was blocked by tFAW for rank r	Characterization: tFAW pressure
OBS_TWTR_HITS (R)	Incremented when RD was blocked by tWTR	DQ-bus turnaround pressure
OBS_XRANK_SWITCHES (R)	Incremented on every last_cmd_rank change (NR>1)	Rank-locality telemetry

The cross-rank counters are only present at NUM_RANKS > 1 builds; at single-rank they tie to zero and read 0.

15.12 Verification Notes (cocotb test plan)

Scenario	What it proves
Two ACTs to different banks of same rank, gap = tRRD - 1 cycle	Second ACT blocks; blocks_act[r][*] was high
Two ACTs to different banks of same rank, gap = tRRD cycles	Second ACT issues at exactly tRRD
Five ACTs to rank 0 within tFAW window	Fifth blocks; tFAW gate engages
Five ACTs spaced across tFAW boundary (4 in + 1 out)	Fifth issues; tFAW slot 1 has aged out
Multi-rank: ACTs on rank 0 do NOT block ACTs on rank 1	Per-rank tRRD / tFAW isolation
Multi-rank: RD to rank 0 then RD to rank 1, gap < tRTRS	Second blocks; blocks_rd[1][*] was high
Multi-rank: RD to rank 0, then PRE to rank 0, then RD to rank 1	Intervening PRE clears tRTRS gate
WR last beat → RD before tWTR elapsed	RD blocks; tWTR gate engaged
RD last beat → WR before tRTW elapsed (DDR3+ scenario)	WR blocks; tRTW gate engaged

Scenario	What it proves
Multi-rank with tCS > 0: cross-rank ACT blocks for tCS cycles	tCS gate works
`NUM_RANKS = 1` build: cross-rank gates synthesize to 0	Synthesis sanity
Heavy multi-rank traffic: telemetry counter increments correctly	CSR observability

15.13 Open Questions / Future Work

- **tRTW cross-rank rank-switch gate.** Currently `cross_rank_block_wr` only depends on tCS (any rank switch), not on a separate "tRTRS-equivalent for writes." DDR4 introduced this; for DDR2/3 it's typically subsumed by tCS + tRTW. Worth a verification scenario at DDR3+ to confirm.
- **tFAW slot-selection policy.** "Pick smallest cnt" works correctly but doesn't necessarily match the literal "evict oldest by arrival." For a 4-entry pool with same-cycle arrivals, the two are equivalent. If multiple ACTs hit in the same cycle (single-issue scheduler means this doesn't currently happen), behavior would need re-spec.
- **Per-rank tCCD?** DDR4 introduced bank groups, with same-group tCCD_L being longer than different-group tCCD_S. For DDR2/LPDDR2 this is not relevant (no bank groups). The DDR3+ family controllers will add a per-bank-group tCCD tracker.
- **Counter-shift vs flop-pool for tFAW.** An alternative implementation is a single shift register that ratchets a tFAW-cycles delay line. Saves one comparator but consumes the same flops. Not worth changing in v1; revisit if Synth flags the comparator chain.

16 Refresh Manager (`refresh_mgr_fub`)

Module: `refresh_mgr_fub.sv` **Location:** `rtl/fub/` **Status:** Skeleton — to be authored during MAS week

> Round-robin rank dispatch, DARP bank selection, postponer, periodic ZQCS, per-rank PASR mask. See HAS §3.4.

16.1 TODO (Week-of-MAS work)

- Interface signal list (inputs/outputs, widths, parameter dependencies)
- Internal state and storage
- FSM diagram (mermaid in `assets/mermaid/`)
- Datapath timing and per-stage flops
- Timing budget on critical paths
- CSR hooks (observability registers driven by this FUB)
- Verification notes (cocotb test plan)

---

17 Init Engine (`init_engine_fub`)

Module: `init_engine_fub.sv` **Location:** `rtl/fub/` **Status:** Skeleton — to be authored during MAS week

> Microprogram-driven cold-boot. Step-table ROM in `rtl/macro/` per memtype. See HAS §3.5.

17.1 TODO (Week-of-MAS work)

- Interface signal list (inputs/outputs, widths, parameter dependencies)
- Internal state and storage
- FSM diagram (mermaid in `assets/mermaid/`)
- Datapath timing and per-stage flops
- Timing budget on critical paths
- CSR hooks (observability registers driven by this FUB)

- Verification notes (cocotb test plan)

---

18 Power-State FSM (`power_state_fub`)

Module: `power_state_fub.sv` **Location:** `rtl/fub/` **Status:** Skeleton — to be authored during MAS week

> Active / Active-PD / Self-Refresh / Deep-PD; per-rank CKE; SR coordination; quiet-point detector. See HAS §3.5.

18.1 TODO (Week-of-MAS work)

- Interface signal list (inputs/outputs, widths, parameter dependencies)
- Internal state and storage
- FSM diagram (mermaid in `assets/mermaid/`)
- Datapath timing and per-stage flops
- Timing budget on critical paths
- CSR hooks (observability registers driven by this FUB)
- Verification notes (cocotb test plan)

---

19 Command Encoder (`cmd_encoder_fub`)

Module: `cmd_encoder_fub.sv` **Location:** `rtl/fub/` **Status:** Skeleton — to be authored during MAS week

> DDR2 ras/cas/we vs LPDDR2 20-bit CA bus, selected at elaboration by `MEMTYPE`. See HAS §3.6.

19.1 TODO (Week-of-MAS work)

- Interface signal list (inputs/outputs, widths, parameter dependencies)
- Internal state and storage
- FSM diagram (mermaid in `assets/mermaid/`)
- Datapath timing and per-stage flops
- Timing budget on critical paths
- CSR hooks (observability registers driven by this FUB)
- Verification notes (cocotb test plan)

---

20 DFI Gear (`gear_dfi_fub`)

Module: `gear_dfi_fub.sv` **Location:** `rtl/fub/` **Status:** Skeleton — to be authored during MAS week

> Per-phase packing per `N_PHASES` / `WRPHASE` / `RDPHASE`. See HAS §3.6 `gear_dfi`.

20.1 TODO (Week-of-MAS work)

- Interface signal list (inputs/outputs, widths, parameter dependencies)
- Internal state and storage
- FSM diagram (mermaid in `assets/mermaid/`)
- Datapath timing and per-stage flops
- Timing budget on critical paths
- CSR hooks (observability registers driven by this FUB)
- Verification notes (cocotb test plan)

---

21 ODT Control (odt_ctrl_fub)

Module: odt_ctrl_fub.sv **Location:** rtl/fub/ **Status:** Skeleton — to be authored during MAS week

Per-rank dfi_odt[r] per ODT_RULE_MULTIRANK. The home of the cross-rank read/write ODT rules. See HAS §3.6.

21.1 TODO (Week-of-MAS work)

- Interface signal list (inputs/outputs, widths, parameter dependencies)
- Internal state and storage
- FSM diagram (mermaid in assets/mermaid/)
- Datapath timing and per-stage flops
- Timing budget on critical paths
- CSR hooks (observability registers driven by this FUB)
- Verification notes (cocotb test plan)

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

22 Write Data Path (wr_data_path_fub)

Module: wr_data_path_fub.sv **Location:** rtl/fub/ **Status:** Skeleton — to be authored during MAS week

AXI W → DFI wrdata. Width conversion AXI DW → NP × DFI_DW. Strobe propagation. See HAS §3.7.

22.1 TODO (Week-of-MAS work)

- Interface signal list (inputs/outputs, widths, parameter dependencies)
- Internal state and storage
- FSM diagram (mermaid in assets/mermaid/)
- Datapath timing and per-stage flops
- Timing budget on critical paths
- CSR hooks (observability registers driven by this FUB)

- Verification notes (cocotb test plan)

---

23 Read Data Path (`rd_data_path_fub`)

Module: `rd_data_path_fub.sv` **Location:** `rtl/fub/` **Status:** Skeleton — to be authored during MAS week

> DFI rddata → AXI R. CL/CWL-aware capture, ID-tagged OoO completion.
> See HAS §3.7.

23.1 TODO (Week-of-MAS work)

- Interface signal list (inputs/outputs, widths, parameter dependencies)
- Internal state and storage
- FSM diagram (mermaid in `assets/mermaid/`)
- Datapath timing and per-stage flops
- Timing budget on critical paths
- CSR hooks (observability registers driven by this FUB)
- Verification notes (cocotb test plan)

---

24 AXI4 Slave Protocol

Status: Skeleton — to be authored during MAS week

The AXI4 slave port carries host requests into the controller.

24.1 Scope of This Section

This section is the **wire-level** AXI4 contract specific to this controller — what's supported, what's relaxed, what's omitted. It is not an AXI4 tutorial; refer to ARM IHI 0022 for the full protocol.

24.2 Supported Features (v1)

- AW/W/B/AR/R 5-channel handshake
- INCR burst type (mandatory)
- ID-aware OoO completion (when `AXI_OOO_ACROSS_IDS = true`)
- 64 / 128 / 256-bit data widths
- 1–16-bit ID widths
- Per-ID in-order completion

24.3 Optional Features (build-time)

- FIXED burst (`SUPPORT_FIXED_BURST`)
- WRAP burst (`SUPPORT_WRAP_BURST`)

24.4 Unsupported Features

- AXI exclusive accesses
- AXI4 user signals (no `USER_*` widths)
- AXI4 cache-coherent extensions

24.5 TODO

- Per-channel timing diagrams (wavedrom)
- Backpressure semantics in detail
- Response-ordering examples for OoO-across-IDs
- Burst-boundary corner cases (4 KB boundary, address-alignment)

---

25 DFI v2.1 Master Protocol

Status: Skeleton — to be authored during MAS week

The DFI v2.1 master port drives the DRAM PHY.

25.1 Scope

This section is the wire-level DFI v2.1 contract as implemented by this controller. The full DFI v2.1 spec covers more — training, frequency change, low-power negotiation — see HAS §2.1 for what is and isn't in scope here.

25.2 Sub-Interfaces Supported

- Command sub-interface
- Write-data sub-interface
- Read-data sub-interface
- Status sub-interface
- Update sub-interface (controller-initiated and PHY-initiated)

25.3 Sub-Interfaces Not Supported (v1)

- Training (not required at v2.1 for DDR2/LPDDR2)
- Frequency change
- Low-power negotiation (CKE used directly)
- CRC (not in DDR2/LPDDR2)

25.4 Phase / Gear Topology

`N_PHASES` per MC clock; gear FUB (see §2.15) handles packing.

25.5 TODO

- Per-signal timing diagrams (rise-to-rise)
- WRPHASE / RDPHASE selection guidelines
- ctrlupd / phyupd protocol detail (rare in v2.1 but spec-mandated)
- DDR2 ras/cas/we vs LPDDR2 CA encoding (cross-ref to §2.14)

26 APB CSR Slave Protocol

Status: Skeleton — to be authored during MAS week

The APB CSR slave is the configuration and observation interface.

26.1 Protocol Compliance

APB3 (`pslverr` supported); APB4 (`pstrb`) optional.

26.2 Behavior

- Two-cycle access (SETUP + ENABLE) per the APB spec
- `pslverr` returns 1 for:
  - Writes to unimplemented schemes in `ADDR_MAP_TUNING.scheme_or`
  - Writes to ranks beyond `NUM_RANKS` (PASR registers, observation registers)
  - Writes to read-only registers
  - Reads from reserved addresses
- All registers are 32-bit; sub-32-bit accesses are aligned to 32-bit boundaries

26.3 CDC

The APB↔MC CDC lives in this FUB. See §1.3 and §2.18 `csr_apb_fub` for the CDC topology.

26.4 TODO

- pslverr decision tree
- APB-side timing diagram
- Sub-32-bit access policy
- CDC implementation detail

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

27 Register Map

Status: Skeleton — to be authored during MAS week

The full CSR map is in HAS §6.3. This MAS section is the **implementation-level** view: how each register is generated, what the reset value generation rule is, how runtime overrides are staged, and the canonical CSR YAML source.

27.1 Source of Truth

The CSR map is generated from `regs/ddr2_lpddr2_csrs.yaml` (PeakRDL- or RDL-style YAML). The build runs the generator to produce:

- `regs/ddr2_lpddr2_csr_pkg.sv` — SV package with offsets, masks, reset values
- `regs/ddr2_lpddr2_csr_block.sv` — the wrapped register file
- `regs/ddr2_lpddr2_csrs.h` — C header for software
- `regs/ddr2_lpddr2_csr_block_uvm.sv` — UVM register model

The MAS does not re-publish the bit-level register table; it documents the **generation pipeline** and the **runtime semantics** (quiet-point staging, etc.).

27.2 Staging vs. Live State

Every R/W field has two storage cells:

1. **APB-side staging register** — written immediately on the APB write
2. **MC-side live register** — copied from staging on the next quiet point

The staging→live commit is one MC cycle. Reads can return either side depending on the field — observation fields (`OBS_*`) read live state; configuration fields (`*_TUNING`) read staging by default but switch to live after `CTRL.config_apply` is acknowledged.

27.3 TODO

- CSR YAML schema reference
- Generation script invocation
- Quiet-point handshake protocol detail
- Software access pattern (config-apply sequence)
- Generated UVM register model overview

28 Runtime Overrides and Quiet Points

Status: Skeleton — to be authored during MAS week

This section documents the runtime override mechanism — how a CSR write becomes a live datapath change without rebuilding or resetting.

28.1 Override Categories

Three flavors of override exist:

1. **Synthesized-MAX, runtime-active** — the silicon includes the maximum-capability logic; runtime CSR selects how much of it is active. Examples: `SCHED_TUNING.lookahead_active`, `REFRESH_TUNING.refresh_defer_active`.
2. **Synthesized-set, runtime-mux** — the silicon includes a discrete set of variants; runtime CSR picks one. Examples: `ADDR_MAP_TUNING.scheme_or`, `REFRESH_TUNING.refpb_policy_or`.
3. **Pure runtime** — no silicon-area cost beyond the register itself. Examples: `REFRESH_TUNING.zqcs_freq_hz`, `INIT_TUNING.zq_retries`.

28.2 Quiet-Point Sequence

1. Software writes the staging field via APB.
2. Software writes `CTRL.config_apply = 1`.
3. Controller drives `txn_queue` to drain, blocks new AXI accepts, completes any in-flight refresh.
4. `power_state_fub` asserts `quiet_point` for one cycle.
5. `csr_apb_fub` copies staging → live for every R/W field.
6. `STATUS.config_settled` rises.
7. Software reads `STATUS.config_settled`, clears `CTRL.config_apply`, and resumes traffic.

The drain phase is bounded — typically < 256 MC cycles for a healthy queue — and is the only operational disruption from a runtime override.

28.3 TODO

- Worst-case drain latency analysis

- Override propagation timing for each category
- Multi-write coalescing (if multiple staging writes are queued, commit them all at one quiet point)
- Software examples (init-time vs. runtime override patterns)

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

29 Family-Wide Config-Bit Applicability

Status: Skeleton — to be authored during MAS week

> The full family-wide config-bit registry is in HAS §5.4. This MAS section documents the implementation-level shape of those bits in **this controller specifically**.

29.1 Per-Bit Implementation Notes for DDR2 / LPDDR2

Bits flagged `(reserved)` in HAS §5.4 ECC_TUNING are present in the CSR map at the documented offsets but read as zero and writes are ignored. The CSR slave does not raise `pslverr` on these — software portability requires writing the family-wide config word to succeed on flavors that haven't implemented every bit.

Bits flagged `DDR3+`, `DDR4+`, `LPDDR3+`, `LPDDR4-only` likewise read as zero and ignore writes in this DDR2/LPDDR2 controller.

The `0xFF8` capability vector reports the build's actual capabilities; software queries this once at boot to decide which family-wide writes are meaningful.

29.2 Soft-N/A Behavior Examples

- Writing `SCHED_TUNING.bank_group_balance = 1` in DDR2 build: silently ignored, no `pslverr`. Read-back returns 0.
- Writing `POWER_TUNING.dpd_enable = 1` in DDR2 build: silently ignored. (DPD is LPDDR-only.)
- Reading `INIT_TUNING.cbt_enable` in any v1 controller: returns 0. (LPDDR4 only.)

29.3 TODO

- Per-bit implementation note for every family-wide bit
- Test plan for each soft-N/A bit
- Software bring-up sequence demonstrating capability-vector-gated writes

---

30 Initialization Sequence

Status: Skeleton — to be authored during MAS week

The init sequence from power-on through DRAM ready, as seen by software.

30.1 TODO

- Step-by-step software pseudocode
- Timing of each step (CSR write → controller state advance)
- Error-detection at each step
- Multi-rank init differences (per-rank ZQ, per-rank MR loads)
- Sim shortcut path (`SIM_INIT_SCALE > 1`)

---

31 Refresh and Power-State Programming

Status: Skeleton — to be authored during MAS week

Programming the refresh subsystem and the power-state FSM from software.

31.1 TODO

- Setting tREFI from CSR
- Selecting REFab vs REFpb per rank

- LPDDR2 PASR mask programming per rank
- Temperature-derate read-back (MR4 / TEMP_DERATE_RANK)
- ZQCS frequency tuning
- Entering / exiting self-refresh
- Entering / exiting Deep Power Down (LPDDR2)

---

32 Multi-Rank Programming

Status: Skeleton — to be authored during MAS week

The multi-rank-specific programming concerns: per-rank MR loads, per-rank PASR, cross-rank ODT, refresh fairness.

32.1 TODO

- Per-rank MR0/1/2/3 programming sequence
- Per-rank PASR programming
- ODT rule selection (`ODT_RULE_MULTIRANK`)
- Cross-rank ODT verification with the BFM
- Rank-disable semantics (`RANK_TUNING.rank_enable_mask`)
- Multi-rank refresh schedule observation

---

33 Error Handling

Status: Skeleton — to be authored during MAS week

Error reporting paths and the software response model.

33.1 TODO

- Error categories: init failure, refresh-deadline miss, AXI queue overflow, DFI rddata-valid timeout
- IRQ topology (`irq_init_done`, `irq_init_error`, `irq_overflow`)
- Clearing latched errors via CSR
- Diagnosis sequence: history register (`STATUS_HISTORY`), debug observation FUBs
- Software recovery patterns (re-init, soft-reset, full reset)

---

34 Build-Time Configuration Reference

Status: Skeleton — to be authored during MAS week

> The full build-time parameter table is in HAS §5.2. This MAS section is the **build-script** view: how each parameter is wired through filelists, includes, and synthesis scripts.

34.1 TODO

- Parameter override on synthesis command line
- Filelist composition per parameter combination
- `rtl/includes/ddr2_lpddr2_config.svh` macro definitions
- Conditional FUB inclusion (e.g., `page_predictor` only when `PAGE_POLICY == HAPPY_HYBRID`)
- Per-memtype filelists

---

35 Runtime Configuration Reference

Status: Skeleton — to be authored during MAS week

The full runtime CSR is in HAS §6.3. This MAS section is the **driver-author** view: which fields can be tuned at runtime, in what order, and with what side-effects.

35.1 TODO

- Field-by-field runtime tuning cookbook
- Recommended bring-up sequence
- Recommended characterization sweep sequence
- Telemetry registers driving each tuning decision